

Formulación. La formulación de programación dinámica para este problema es

Etapla $n = n$ -ésima jugada del juego ($n = 1, 2, 3$),

x_n = número de fichas que debe apostar en la etapa n ,

Etapla s_n = número de fichas disponibles para comenzar la etapa n .

Se eligió esta definición del estado del sistema porque proporciona la información necesaria sobre la situación actual para tomar una decisión óptima sobre cuántas fichas apostar en la siguiente jugada.

Como el objetivo es maximizar la probabilidad de que la joven gane la apuesta, la función objetivo que debe maximizarse en cada etapa es la probabilidad de terminar las tres jugadas con cinco fichas o más. (Observe que el valor de terminar con más de cinco fichas es el mismo que el de terminar con cinco fichas, ya que la apuesta se gana de las dos formas.) Así,

$f_n(s_n, x_n)$ = probabilidad de terminar las tres jugadas con cinco fichas o más, dado que la joven comienza la etapa n con s_n fichas, su decisión inmediata es x_n y en adelante toma decisiones óptimas,

$$f_n^*(s_n) = \max_{x_n=0, 1, \dots, s_n} f_n(s_n, x_n).$$

La expresión para $f_n(s_n, x_n)$ debe reflejar el hecho de que, en un momento dado, es posible acumular cinco fichas aun cuando pierda la siguiente jugada. Si pierde, el estado en la siguiente etapa será $s_n - x_n$, y la probabilidad de terminar con al menos cinco fichas será entonces $f_{n+1}^*(s_n - x_n)$. Por el contrario, si gana la siguiente jugada, el estado del sistema será $s_n + x_n$ y la probabilidad correspondiente será $f_{n+1}^*(s_n + x_n)$. Como se supone que la probabilidad de ganar una jugada dada es $\frac{2}{3}$, se concluye que

$$f_n(s_n, x_n) = \frac{1}{3} f_{n+1}^*(s_n - x_n) + \frac{2}{3} f_{n+1}^*(s_n + x_n)$$

[donde $f_4^*(s_4)$ se define como 0 para $s_4 < 5$ y 1 para $s_4 \geq 5$]. Entonces, no existe una contribución directa a la función objetivo por parte de la etapa n además del efecto que tiene estar en el siguiente estado. En la figura 10.13 se resumen estas relaciones básicas.

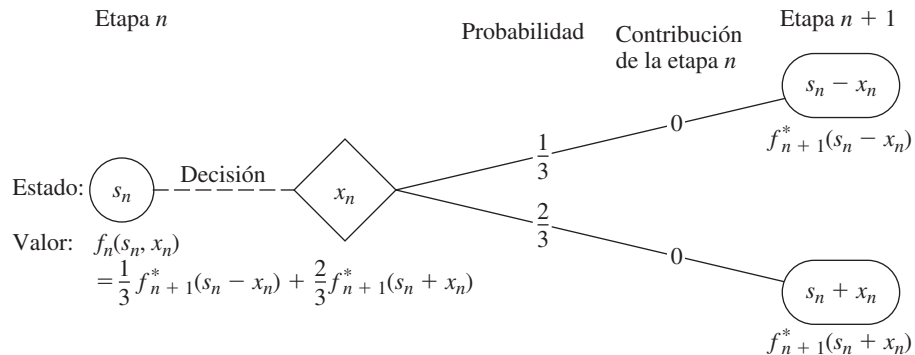
Entonces, la relación recursiva para este problema es

$$f_n^*(s_n) = \max_{x_n=0, 1, \dots, s_n} \left\{ \frac{1}{3} f_{n+1}^*(s_n - x_n) + \frac{2}{3} f_{n+1}^*(s_n + x_n) \right\},$$

para $n = 1, 2, 3$, con $f_4^*(s_4)$ como se definió.

Procedimiento de solución. Esta relación recursiva conduce a los siguientes resultados de cálculos.

■ **FIGURA 10.13**
Estructura básica del problema de Las Vegas.



$n = 3:$	s_3	$f_3^*(s_3)$	x_3^*
	0	0	—
	1	0	—
	2	0	—
	3	$\frac{2}{3}$	2 (o más)
	4	$\frac{2}{3}$	1 (o más)
	≥ 5	1	0 (o $\leq s_3 - 5$)

$n = 2:$	s_2	x_2	$f_2(s_2, x_2) = \frac{1}{3}f_3^*(s_2 - x_2) + \frac{2}{3}f_3^*(s_2 + x_2)$					$f_2^*(s_2)$	x_2^*
			0	1	2	3	4		
	0	0					0	—	
	1	0	0				0	—	
	2	0	$\frac{4}{9}$	$\frac{4}{9}$			$\frac{4}{9}$	1 o 2	
	3	$\frac{2}{3}$	$\frac{4}{9}$	$\frac{2}{3}$	$\frac{2}{3}$		$\frac{2}{3}$	0, 2 o 3	
	4	$\frac{2}{3}$	$\frac{8}{9}$	$\frac{2}{3}$	$\frac{2}{3}$	$\frac{2}{3}$	$\frac{8}{9}$	1	
≥ 5	1					1	0 (o $\leq s_2 - 5$)		

$n = 1:$	s_1	x_1	$f_1(s_1, x_1) = \frac{1}{3}f_2^*(s_1 - x_1) + \frac{2}{3}f_2^*(s_1 + x_1)$				$f_1^*(s_1)$	x_1^*
			0	1	2	3		
	3		$\frac{2}{3}$	$\frac{20}{27}$	$\frac{2}{3}$	$\frac{2}{3}$	$\frac{20}{27}$	1

Por tanto, la política óptima es

$$x_1^* = 1 \begin{cases} \text{si gana,} & x_2^* = 1 \begin{cases} \text{si gana,} & x_3^* = 0 \\ \text{si pierde,} & x_3^* = 2 \text{ o } 3. \end{cases} \\ \text{si pierde,} & x_2^* = 1 \text{ o } 2 \begin{cases} \text{si gana,} & x_3^* = \begin{cases} 2 \text{ o } 3 & (\text{para } x_2^* = 1) \\ 1, 2, 3 \text{ o } 4 & (\text{para } x_2^* = 2) \end{cases} \\ \text{si pierde,} & \text{la apuesta está perdida} \end{cases} \end{cases}$$

Esta política da a la experta en estadística una probabilidad de $\frac{20}{27}$ de ganar la apuesta a sus colegas.

10.5 CONCLUSIONES

La programación dinámica es una técnica muy útil para tomar una *sucesión de decisiones interrelacionadas*. Requiere la formulación de una *relación recursiva* apropiada para cada problema individual. Sin embargo, proporciona grandes ahorros computacionales en comparación con la enumeración exhaustiva para encontrar la mejor combinación de decisiones, en especial cuando se trata de problemas grandes. Por ejemplo, si un problema tiene 10 etapas con 10 estados y 10 decisiones posibles en cada etapa, la enumeración exhaustiva tendría que considerar hasta 10 mil millones de combinaciones, mientras que la programación dinámica necesita hacer como máximo mil cálculos (10 para cada estado en cada etapa).

En este capítulo se presentó sólo programación dinámica con un número *finito* de etapas. El capítulo 19 está dedicado a un tipo general de modelos de programación dinámica probabilista en donde las etapas continúan de manera indefinida, tema conocido como procesos markovianos de decisión.

■ REFERENCIAS SELECCIONADAS

1. Bertsekas, D. P., *Dynamic Programming: Deterministic and Stochastic Models*, Prentice-Hall, Englewood Cliffs, NJ, 1987.
2. Denardo, E. V., *Dynamic Programming Theory and Applications*, Prentice-Hall, Englewood Cliffs, NJ, 1982.
3. Howard, R. A., "Dynamic Programming", en *Management Science*, **12**: 317-345, 1966.
4. Lew, A. y H. Mauch: *Dynamic Programming: A Computational Tool*, Springer, Nueva York, 2007.
5. Smith, D. K., *Dynamic Programming: A Practical Introduction*, Ellis Horwood, Londres, 1991.
6. Sniedovich, M., *Dynamic Programming*, Marcel Dekker, Nueva York, 1991.

■ AYUDAS DE APRENDIZAJE PARA ESTE CAPÍTULO EN EL SITIO EN INTERNET DEL LIBRO (www.mhhe.com/hillier)

Ejemplos resueltos:

Ejemplos para el capítulo 10

Archivo de LINGO "Cap. 10. Dynamic Programming"

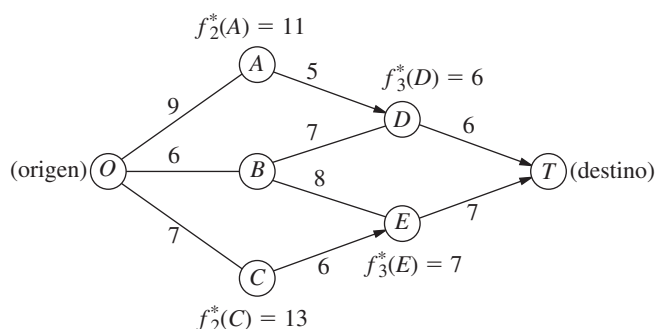
Glosario del capítulo 10

Vea el apéndice 1 para la documentación del software.

■ PROBLEMAS

Un asterisco en el número del problema indica que al final del libro se proporciona al menos una respuesta parcial.

10.2-1. Considere la siguiente red en la que cada número junto a una trayectoria representa la distancia real entre el par de nodos que conecta. El objetivo es encontrar la ruta más corta del origen al destino.



- a) ¿Cuáles son las etapas y los estados para la formulación de programación dinámica de este problema?
- b) Utilice programación dinámica para resolver el problema, pero en lugar de emplear las tablas usuales muestre el trabajo en una gráfica (similar a la figura 10.2). En particular, comience con la red dada, en la que se proporcionan los valores de $f_n^*(s_n)$ de cuatro de los nodos; después encuentre $f_2^*(B)$ y $f_1^*(O)$. Dibuje una punta de flecha para indicar la trayectoria óptima que debe tomarse al

salir de estos últimos dos nodos. Por último, identifique la ruta óptima con las flechas desde el nodo O hasta el nodo T .

- c) Utilice programación dinámica para resolver este problema; construya a mano las tablas usuales para $n = 3$, $n = 2$ y $n = 1$.
- d) Utilice el algoritmo de la ruta más corta que se presentó en la sección 9.3 para resolver este problema. Compare este enfoque con el de los incisos b) y c).

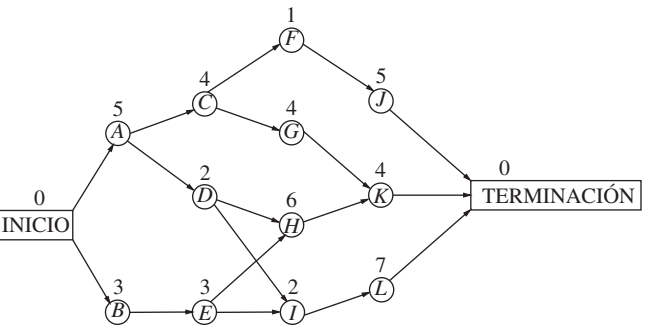
10.2-2. El gerente de ventas de una editorial de libros de texto universitarios tiene seis agentes de ventas que puede asignar a tres regiones distintas del país. Ha decidido que cada región debe tener por lo menos un agente y que cada uno de éstos debe quedar restringido a una de estas regiones, pero ahora quiere determinar cuántos agentes debe asignar a las respectivas regiones con el fin de maximizar las ventas.

La tabla de la parte superior de la siguiente columna da el incremento estimado de las ventas en cada región (en las unidades apropiadas) si se le asignan diferentes cantidades de agentes:

Agente de ventas	Región		
	1	2	3
1	40	24	32
2	54	47	46
3	78	63	70
4	99	78	84

- a) Utilice programación dinámica para resolver este problema. En lugar de usar las tablas normales, muestre su trabajo con una gráfica de una red similar a la del problema 10.2-1. Haga lo mismo que en el problema 10.2-1b para obtener $f_n^*(s_n)$ de cada nodo (excepto el nodo terminal) y escriba sus valores al lado. Dibuje una punta de flecha para indicar la ruta óptima (o rutas en el caso de empates) que debe tomarse al salir de cada nodo. Por último, identifique la ruta (o rutas) óptima(s) que obtuvo a través de la red y la solución (o soluciones) óptima(s) correspondiente.
- b) Utilice programación dinámica para resolver este problema; elabore las tablas normales con $n = 3, n = 2$ y $n = 1$.

10.2-3. Considere la siguiente red de proyecto (como se describe en la sección 9.8), donde el número sobre el nodo es el tiempo que se requiere para la actividad correspondiente. Considere el problema de encontrar la *trayectoria más larga* (el mayor tiempo total) a través de esta red desde el inicio hasta su término, puesto que la trayectoria más larga es la ruta crítica.



- a) ¿Cuáles son las etapas y los estados para formular la programación dinámica de este problema?
- b) Use programación dinámica para resolver este problema, pero en lugar de emplear las tablas usuales, muestre su trabajo en una gráfica. En particular, determine los valores de las distintas $f_n^*(s_n)$ junto a los nodos correspondientes y el arco óptimo que debe tomarse al salir de cada nodo mediante una punta de flecha cerca del inicio del arco. Después identifique la trayectoria óptima (la trayectoria más larga) que siguen estas puntas de flecha desde el nodo inicio hasta el de término. Si existe más de una trayectoria óptima, identifíquelas a todas.
- c) Utilice programación dinámica para resolver el problema con las tablas usuales para $n = 4, n = 3, n = 2$ y $n = 1$.

10.2-4. Considere las siguientes afirmaciones sobre la solución de problemas de programación dinámica. Diga si cada una es falsa o verdadera y después justifique su respuesta con referencia a afirmaciones específicas (con la cita de la página) dentro del capítulo.

- a) El procedimiento de solución utiliza una relación recursiva que permite obtener la política óptima para la etapa $(n + 1)$ dada la política óptima de la etapa n .
- b) Después de completar el procedimiento de solución, si se toma por error una decisión no óptima en alguna etapa, deberá aplicarse de nuevo el procedimiento para determinar las nuevas decisiones óptimas (dada esta decisión no óptima) en las etapas subsecuentes.
- c) Una vez que se ha encontrado una política óptima para todo el problema, la información necesaria para especificar la decisión óptima en una etapa en particular es el estado de esa etapa y las decisiones que se tomaron en las etapas anteriores.

10.3-1. Lea el artículo de referencia que describe en su totalidad el estudio de investigación de operaciones que se resume en el recuadro de aplicación que se presentó en la sección 10.3. Describa de manera breve la forma como se aplicó la programación dinámica en dicho estudio. Después, elabore una lista de los beneficios financieros y de otro tipo que arrojó este estudio.

10.3-2.* El propietario de una cadena de tres supermercados compró cinco cargas de fresas frescas. La distribución de probabilidad estimada de las ventas potenciales de las fresas antes de que se echen a perder difiere entre los tres supermercados. El propietario quiere saber cómo debe asignar las cinco cargas a las tiendas para maximizar la ganancia esperada.

Por razones administrativas, no quiere dividir las cargas entre las tiendas. Sin embargo, está de acuerdo en asignar cero cargas a cualquiera de ellas.

En la siguiente tabla se proporciona la ganancia estimada de cada tienda al asignar distintas cantidades de cargas:

Número de carga	Tienda		
	1	2	3
0	0	0	0
1	5	6	4
2	9	11	9
3	14	15	13
4	17	19	18
5	21	22	20

Utilice programación dinámica para determinar cuántas cargas debe asignarse a cada tienda para maximizar la ganancia total esperada.

10.3-3. Una estudiante universitaria cuenta con siete días para preparar los exámenes finales de cuatro cursos y quiere asignar su tiempo de estudio de la manera más eficiente posible. Necesita por lo menos un día para cada curso y quiere concentrarse sólo en un curso cada día por lo que quiere asignar uno, dos, tres o cuatro días a cada curso. Como hace poco tomó un curso de investigación de operaciones, decide aplicar programación dinámica para hacer estas asignaciones que maximicen el total de puntos obtenidos en los cuatro cursos. Estima que las distintas asignaciones en días de estudio le reditarán puntos de calificación según la siguiente tabla:

Día de estudio	Puntos de calificación estimados			
	Curso			
	1	2	3	4
1	1	5	4	4
2	3	6	6	4
3	6	8	7	5
4	8	8	9	8

Resuelva este problema con programación dinámica.

10.3-4. Una campaña política se encuentra en su última etapa y las preliminares indican que las preferencias electorales se encuentran

sumamente cerradas. Uno de los candidatos tiene suficientes fondos para comprar tiempo de TV por un total de cinco comerciales en las horas de mayor audiencia en estaciones localizadas en cuatro áreas diferentes. Con base en la información de las preliminares se hizo una estimación del número de votos adicionales que se puede ganar en las áreas de difusión según el número de comerciales que se contrate. Estas estimaciones se dan en la tabla en miles de votos:

Comerciales	Área			
	1	2	3	4
0	0	0	0	0
1	4	6	5	3
2	7	8	9	7
3	9	10	11	12
4	12	11	10	14
5	15	12	9	16

Utilice programación dinámica para determinar cómo deben distribuirse los cinco comerciales entre las cuatro áreas con el fin de maximizar el número estimado de votos ganados.

10.3-5. La presidenta de un partido político de un estado planea las próximas elecciones presidenciales. Cuenta con la colaboración de seis voluntarios para trabajar en los distritos electorales y los quiere asignar a cuatro distritos de manera que se maximice su eficacia. Ella piensa que sería ineficiente asignar un voluntario a más de un distrito, pero está dispuesta a no asignar a nadie a cualquiera de ellos si pueden lograr más en otro distrito.

En la siguiente tabla se presenta el aumento estimado del número de votos para el candidato del partido en cada distrito si se asigna distintos números de voluntarios:

Voluntarios	Distrito			
	1	2	3	4
0	0	0	0	0
1	4	7	5	6
2	9	11	10	11
3	15	16	15	14
4	18	18	18	16
5	22	20	21	17
6	24	21	22	18

Este problema tiene varias soluciones óptimas para determinar cuántos voluntarios deben asignarse a cada distrito para maximizar el incremento total estimado de la popularidad del candidato del partido. Utilice programación dinámica para encontrar todas las soluciones óptimas para que la presidenta del partido pueda hacer una selección basada en otros factores.

10.3-6. Utilice programación dinámica para resolver el problema de programación de la producción de Northern Airplane Co., que se presentó en la sección 8.1 (vea la tabla 8.7). Suponga que las cantidades producidas deben ser enteros múltiplos de cinco.

10.3-7.* Una compañía está por introducir un nuevo producto a un mercado muy competido y planea su estrategia de comercialización. Se ha tomado la decisión de introducir el producto en tres fases. La

fase 1 incluye ofertas especiales de introducción a precio reducido para atraer a los compradores de primera vez. La fase 2 es una campaña intensiva de comerciales y anuncios para persuadir a estos compradores de primera vez a que continúen comprando el producto a precio normal. Se sabe que otra compañía introducirá otro nuevo producto competitivo más o menos al terminar la fase 2. En consecuencia, la fase 3 incluye una campaña de seguimiento y promoción para tratar de evitar que los clientes regulares cambien a la competencia.

Se cuenta con un presupuesto total de 4 millones de dólares para realizar esta campaña. El problema consiste en determinar cómo asignar este dinero de la manera más eficaz a las tres fases. Sea m la proporción de mercado inicial (expresada como porcentaje) que se logra en la fase 1, f_2 la fracción de este mercado que se retiene en la fase 2 y f_3 la fracción restante del porcentaje de mercado que se retiene en la fase 3. Con los datos de la siguiente tabla, aplique programación dinámica para determinar la asignación de \$4 millones para maximizar el porcentaje final del mercado para el nuevo producto, es decir, maximizar mf_2f_3 .

a) Suponga que el dinero se debe gastar en cantidades enteras múltiplos de 1 millón de dólares en cada fase y que el mínimo permisible es 1 para la fase 1 y 0 para las fases 2 y 3. En la siguiente tabla se proporciona el efecto estimado de los gastos en cada fase:

Millones de dólares gastados	Efecto sobre el porcentaje del mercado		
	m	f_2	f_3
0	—	0.2	0.3
1	20	0.4	0.5
2	30	0.5	0.6
3	40	0.6	0.7
4	50	—	—

b) Suponga que se puede gastar *cualquier* cantidad del presupuesto en cada fase, y que el efecto estimado al gastar una cantidad x_i (en unidades de millones de dólares) en la fase i ($i = 1, 2, 3$) es

$$m = 10x_1 - x_1^2$$

$$f_2 = 0.40 + 0.10x_2$$

$$f_3 = 0.60 + 0.07x_3.$$

[Sugerencia: Después de obtener en forma analítica las funciones $f_2^*(s)$ y $f_3^*(s)$, obtenga x_1^* de manera gráfica.]

10.3-8. Considere un sistema electrónico con cuatro componentes, cada uno de los cuales debe trabajar para que el sistema funcione. La confiabilidad del sistema se puede mejorar si se instalan varias unidades paralelas en uno o más de los componentes. La siguiente tabla muestra la probabilidad de que los respectivos componentes (indicados como componente 1, 2, 3 y 4) funcionen si constan de una, dos o tres unidades paralelas:

Unidades paralelas	Probabilidad de funcionamiento			
	Componente 1	Componente 2	Componente 3	Componente 4
1	0.5	0.6	0.7	0.5
2	0.6	0.7	0.8	0.7
3	0.8	0.8	0.9	0.9

La probabilidad de que el sistema funcione es el producto de las probabilidades de que los componentes respectivos funcionen.

En la siguiente tabla se presenta el costo (en cientos de dólares) de instalar una, dos o tres unidades paralelas en los componentes (indicados como componente 1, 2, 3 y 4) respectivos:

Unidades paralelas	Costo			
	Componente 1	Componente 2	Componente 3	Componente 4
1	1	2	1	2
2	2	4	3	3
3	3	5	4	4

Dadas las limitaciones de presupuesto, se puede gastar un máximo de 1 000 dólares.

Use programación dinámica para determinar cuántas unidades paralelas debe instalar en cada uno de los cuatro componentes para maximizar la probabilidad de que el sistema funcione.

10.3-9. Considere el siguiente problema de programación no lineal entera.

$$\text{Maximizar } Z = 3x_1^2 - x_1^3 + 5x_2^2 - x_2^3,$$

sujeta a

$$x_1 + 2x_2 \leq 4$$

y

$$x_1 \geq 0, \quad x_2 \geq 0$$

x_1, x_2 son enteros.

Use programación dinámica para resolver este problema.

10.3-10. Considere el siguiente problema de programación no lineal entera.

$$\text{Maximizar } Z = 32x_1 - 2x_1^2 + 30x_2 + 20x_3,$$

sujeta a

$$3x_1 + 7x_2 + 5x_3 \leq 20$$

y

$$x_1, x_2, x_3 \text{ son enteros no negativos.}$$

Use programación dinámica para resolver el problema.

10.3-11.* Considere el siguiente problema de programación no lineal.

$$\text{Maximizar } Z = 36x_1 + 9x_1^2 - 6x_1^3 + 36x_2 - 3x_2^3,$$

sujeta a

$$x_1 + x_2 \leq 3$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

Use programación dinámica para resolver el problema.

10.3-12. Resuelva el problema de programación del nivel de empleados de Local Job Shop (ejemplo 4) bajo el supuesto de que el costo total de cambiar el nivel de empleados de una temporada a la siguiente es de \$100 multiplicado por el cuadrado de la diferencia de niveles.

10.3-13. Considere el siguiente problema de programación no lineal.

$$\text{Maximizar } Z = 2x_1^2 + 2x_2 + 4x_3 - x_3^2$$

sujeta a

$$2x_1 + x_2 + x_3 \leq 4$$

y

$$x_1 \geq 0, \quad x_2 \geq 0, \quad x_3 \geq 0.$$

Use programación dinámica para resolver el problema.

10.3-14. Considere el siguiente problema de programación no lineal.

$$\text{Minimizar } Z = x_1^4 + 2x_2^2$$

sujeta a

$$x_1^2 + x_2^2 \geq 2.$$

(Sin restricciones de no negatividad.) Utilice programación dinámica para resolver este problema.

10.3-15. Considere el siguiente problema de programación no lineal.

$$\text{Maximizar } Z = x_1^3 + 4x_2^2 + 16x_3,$$

sujeta a

$$x_1x_2x_3 = 4$$

y

$$x_1 \geq 1, \quad x_2 \geq 1, \quad x_3 \geq 1.$$

a) Utilice programación dinámica para resolver este problema cuando, además de las restricciones dadas, se requiere que todas las variables sean enteras.

b) Utilice programación dinámica para resolver el problema como se estableció (variables continuas).

10.3-16. Considere el siguiente problema de programación no lineal.

$$\text{Maximizar } Z = x_1(1 - x_2)x_3,$$

sujeta a

$$x_1 - x_2 + x_3 \leq 1$$

y

$$x_1 \geq 0, \quad x_2 \geq 0, \quad x_3 \geq 0.$$

Use programación dinámica para resolver el problema.

10.3-17. Considere el siguiente problema de programación lineal.

$$\text{Maximizar } Z = 15x_1 + 10x_2,$$

sujeta a

$$\begin{aligned}x_1 + 2x_2 &\leq 6 \\ 3x_1 + x_2 &\leq 8\end{aligned}$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

Use programación dinámica para resolver el problema.

10.3-18. Considere el siguiente problema de “costo fijo”.

$$\text{Maximizar } Z = 3x_1 + 7x_2 + 6f(x_3),$$

sujeta a

$$\begin{aligned}x_1 + 3x_2 + 2x_3 &\leq 6 \\ x_1 + x_2 &\leq 5\end{aligned}$$

y

$$x_1 \geq 0, \quad x_2 \geq 0, \quad x_3 \geq 0,$$

donde

$$f(x_3) = \begin{cases} 0 & \text{si } x_3 = 0 \\ -1 + x_3 & \text{si } x_3 > 0. \end{cases}$$

Use programación dinámica para resolver el problema.

10.4-1. Un aficionado al backgamon jugará tres encuentros consecutivos con sus amigos. En cada juego tendrá la oportunidad de apostar; la cantidad apostada puede ascender a *cualquier* cantidad entre cero y la cantidad de dinero que le quede después de las apuestas en los juegos anteriores. En cada encuentro, la probabilidad de que gane el juego y, por lo tanto, la apuesta, es $\frac{1}{2}$, mientras que tiene una probabilidad de $\frac{1}{2}$ de perder y, por tanto, perder la apuesta. Comenzará con \$75 y su meta es tener \$100 al finalizar los tres juegos. (No intenta acabar con más de \$100 porque se trata de un encuentro amistoso.) El jugador quiere encontrar la política óptima de apuesta —incluidos todos los empates— que maximice la probabilidad de tener exactamente \$100 después de los tres juegos.

Utilice programación dinámica para resolver este problema.

10.4-2. Imagine que tiene \$10 000 para invertir y que tendrá la oportunidad de hacerlo en cualesquiera de dos inversiones (*A* o *B*) al principio de cada uno de los próximos tres años. Existe incertidumbre respecto del rendimiento de ambas inversiones. Si invierte en *A*, puede perder todo el dinero u (con probabilidad más alta) obtener \$20 000 (una ganancia de \$10 000) al final del año. Si invierte en *B*, puede obtener los mismos \$10 000 o (con probabilidad más baja) \$20 000 al terminar el año. Las probabilidades para que sucedan estos eventos son las siguientes:

Inversión	Cantidad obtenida (\$)	Probabilidad
A	0	0.25
	20 000	0.75
B	10 000	0.9
	20 000	0.1

Se le permite hacer (a lo sumo) *una* inversión al año y sólo puede invertir \$10 000 cada vez. (Cualquier cantidad adicional de dinero acumulada es inútil.)

- Utilice programación dinámica para encontrar la política de inversión que maximice la cantidad de dinero que espera tener después de los tres años.
- Utilice programación dinámica para encontrar la política de inversión que maximice la probabilidad de tener por lo menos \$20 000 después de los 3 años.

10.4-3.* Suponga que la situación del problema de la Hit-and-Miss Manufacturing Company (ejemplo 6) ha cambiado ligeramente. Después de un análisis más cuidadoso, se estima que cada artículo producido tiene una probabilidad de $\frac{2}{3}$, de ser aceptable, en lugar de $\frac{1}{2}$, de manera que la probabilidad de producir *cero* artículos aceptables en un lote de tamaño L es $(\frac{1}{3})^L$. Aún más, ahora sólo se tiene tiempo para hacer dos corridas de producción. Utilice programación dinámica para determinar la nueva política óptima para manejar este problema.

10.4-4. Reconsidere el ejemplo 7. Suponga que se cambia la apuesta a “si comienza con dos fichas, ella no tendrá cinco fichas después de cinco jugadas”. Sin perder de vista los resultados que obtuvo antes, efectúe los cálculos adicionales para determinar cuál es la nueva política óptima para la joven experta en estadística.

10.4-5. El producto más importante de la Profit & Gambit Co., ha perdido dinero por un decremento de sus ventas. De hecho, durante el trimestre actual las ventas estarán 4 millones de unidades abajo del punto de equilibrio. Debido a que el rendimiento marginal por cada unidad vendida es \$5 mayor que el costo marginal, la empresa sufrirá una pérdida de \$20 millones en el trimestre. La administración debe tomar medidas expeditas para rectificar esta situación. Se tiene en consideración dos alternativas. Una consiste en abandonar el producto de inmediato, con lo que se incurriría en un costo de \$20 millones por cierre. La otra es emprender una campaña publicitaria intensiva para aumentar las ventas y después abandonar el producto (a un costo de \$20 millones) sólo si la campaña no tiene el éxito suficiente. Se formularon y analizaron planes tentativos para esta campaña que se extendería a los próximos tres trimestres —sujeta a cancelación en cualquier momento—, cuyo costo sería de \$30 millones por trimestre. Se estima que el aumento aproximado en ventas sería de 3 millones de unidades en el primer trimestre, otros 2 millones en el segundo y otro millón en el tercer trimestre. Sin embargo, por algunas variables impredecibles del mercado, existe gran incertidumbre en cuanto al efecto real que tendrá la campaña: un análisis cuidadoso indica que la estimación de cada trimestre puede estar equivocada hasta en 2 millones de unidades en cualquier dirección. (Para cuantificar la incertidumbre, suponga que los aumentos adicionales de ventas de los tres trimestres son variables aleatorias independientes con distribución uniforme entre 1 y 5 millones, entre 0 y 4 millones y entre -1 y 3 millones, respectivamente.) Si los aumentos reales son demasiado pequeños, se puede interrumpir la campaña publicitaria y discontinuar el producto al final de cualquiera de los dos próximos trimestres.

Si se emprende la campaña publicitaria intensiva y se continúa hasta terminar, se estima que las ventas seguirán en aumento por algún tiempo al mismo nivel que en el tercer trimestre (el último) de la campaña. Por tanto, si las ventas en ese trimestre todavía están por debajo del punto de equilibrio, el producto deberá dejar de producirse. De otra manera, se estima que la ganancia descontada esperada en adelante será de \$40 por cada unidad vendida que esté más arriba del punto de equilibrio en el tercer trimestre.

Utilice programación dinámica para determinar la política óptima que maximice la ganancia esperada.

11

CAPÍTULO

Programación entera

En el capítulo 3 se estudiaron muchos ejemplos de aplicaciones distintas de la programación lineal. Sin embargo, una limitación importante que impide muchas otras aplicaciones es el supuesto de divisibilidad (vea la sección 3.3), que requiere que las variables de decisión puedan tomar valores no enteros. En muchos problemas prácticos, las variables de decisión sólo tienen sentido real si su valor es entero. Por ejemplo, con frecuencia es necesario asignar a las actividades cantidades enteras de personas, máquinas o vehículos. Si el hecho de exigir valores enteros es la única diferencia que tiene un problema con la formulación de programación lineal, entonces se trata de un problema de *programación entera* (PE). (El nombre completo es *programación lineal entera*, pero, por lo general, el adjetivo *lineal* se omite, excepto cuando se hace una comparación con el problema más esotérico de programación no lineal entera que está fuera del alcance de este libro.)

El modelo matemático para programación entera es sencillamente el modelo de programación lineal (vea la sección 3.2) con la restricción adicional de que las variables deben tener valores enteros. Si sólo es necesario que *algunas* de las variables tengan valores enteros —y el supuesto de divisibilidad se cumple para el resto—, el modelo se conoce como de **programación entera mixta** (PEM). Cuando se hace la distinción entre un problema con todas las variables enteras y este caso mixto, el primero se llama de programación entera pura.

Por ejemplo, el problema de la Wyndor Glass Co., que se presentó en la sección 3.1 habría sido en realidad un problema de PE, si las dos variables de decisión x_1 y x_2 hubieran representado el número total de unidades que se debía producir de los respectivos artículos 1 y 2, en lugar de las tasas de producción. Como ambos productos —puertas de vidrio y ventanas con marco de madera— deben ser unidades completas, x_1 y x_2 deben restringirse a valores enteros.

Se realizan numerosas aplicaciones de programación entera que involucran una extensión directa de programación lineal en la que debe eliminarse el supuesto de divisibilidad. Sin embargo, existe otra área de aplicación que puede ser mucho más importante, como el problema que incluye cierto número de “decisiones tipo: sí o no”. En situaciones de este tipo, las únicas dos elecciones posibles son *sí* y *no*. Por ejemplo, ¿debe emprenderse un proyecto específico? ¿Debe hacerse cierta inversión fija? ¿Debe localizarse una instalación en un sitio en particular?

Debido a que estos problemas involucran sólo dos posibilidades, este tipo de decisiones se puede representar mediante variables de decisión restringidas a sólo dos valores, por ejemplo, 0 y 1. De esta forma, la j -ésima decisión sí o no se puede representar por x_j , tal que

$$x_j = \begin{cases} 1 & \text{si la decisión } j \text{ es sí} \\ 0 & \text{si la decisión } j \text{ es no.} \end{cases}$$

Las variables de este tipo se llaman **binarias** (o variables 0-1). En consecuencia, algunas veces se hace referencia a los problemas de programación entera que contienen sólo variables binarias como problemas de **programación entera binaria** (PEB) (o problemas 0-1 de programación entera).

En la sección 11.1 se presenta una versión en miniatura de un problema representativo de PEB y en la sección 11.2 se estudian otras aplicaciones. Otras posibilidades de formulación con

variables binarias se analizan en la sección 11.3, mientras que en la sección 11.4 se presenta una serie de ejemplos de modelos. En las secciones 11.5 a la 11.8 se analizan las maneras de resolver los problemas de PE, que incluye los de PEB y PEM. El capítulo concluye en la sección 11.9, donde se presenta un interesante desarrollo reciente (*programación de restricciones*) que promete expandir en gran forma la capacidad de formular y resolver modelos de programación entera.

11.1 EJEMPLO PROTOTIPO

La CALIFORNIA MANUFACTURING COMPANY analiza la posibilidad de llevar a cabo una expansión mediante la construcción de una nueva fábrica ya sea en Los Ángeles o en San Francisco, o tal vez en ambas ciudades. También piensa en construir, a lo sumo, un nuevo almacén, pero la decisión sobre el lugar en donde lo instalará está restringida a la ciudad donde se construya la nueva fábrica. En la cuarta columna de la tabla 11.1 se muestra el *valor presente neto* —rendimiento total que toma en cuenta el valor del dinero en el tiempo— de cada alternativa. En la última columna se proporciona el capital que se requiere —incluido el valor presente neto— para las respectivas inversiones, donde el capital total disponible es de 10 millones de dólares. El objetivo es encontrar la combinación factible de alternativas que maximice el valor presente neto total.

El modelo PEB

Aun cuando este problema es tan pequeño que se puede resolver con rapidez por inspección —construir fábricas en ambas ciudades, pero ningún almacén—, se formulará el modelo de programación entera con propósitos ilustrativos. Todas las variables de decisión tienen la forma *binaria*

$$x_j = \begin{cases} 1 & \text{si la decisión } j \text{ es sí,} \\ 0 & \text{si la decisión } j \text{ es no,} \end{cases} \quad (j = 1, 2, 3, 4).$$

Sea

Z = valor presente neto de estas decisiones.

Si se hace la inversión para construir una instalación dada —de manera que la variable de decisión correspondiente tenga valor de 1—, el valor presente neto estimado de estas inversiones aparece en la cuarta columna de la tabla 11.1. Si la inversión no se hace —y, por tanto, la variable de decisión es igual a 0—, el valor presente neto es 0. Entonces, con unidades de millones de dólares,

$$Z = 9x_1 + 5x_2 + 6x_3 + 4x_4.$$

La última columna de la tabla 11.1 indica que la cantidad de capital que se gastará en las cuatro instalaciones no puede exceder a 10 millones de dólares. En consecuencia, si se sigue con el uso de estas unidades de millones de dólares, una restricción del modelo es

$$6x_1 + 3x_2 + 5x_3 + 2x_4 \leq 10.$$

Como las últimas dos decisiones representan *alternativas mutuamente excluyentes* —la compañía quiere construir *cundo mucho* un almacén nuevo—, se necesita la restricción

$$x_3 + x_4 \leq 1.$$

■ **TABLA 11.1** Datos del ejemplo de la California Manufacturing Co.

Número de decisión	Pregunta sí o no	Variable de decisión	Valor presente neto	Capital requerido
1	¿Construir la fábrica en Los Ángeles?	x_1	\$9 millones	\$6 millones
2	¿Construir la fábrica en San Francisco?	x_2	\$5 millones	\$3 millones
3	¿Construir el almacén en Los Ángeles?	x_3	\$6 millones	\$5 millones
4	¿Construir el almacén en San Francisco?	x_4	\$4 millones	\$2 millones

Capital disponible: \$10 millones

Aún más, las decisiones 3 y 4 son *contingentes* —o condicionales— porque dependen de las decisiones 1 y 2, respectivamente (la compañía consideraría la construcción de un almacén en determinada ciudad sólo si la nueva fábrica va a estar ahí). Por tanto, en caso de tomar la decisión 3, se requiere que $x_3 = 0$ si $x_1 = 0$. Esta restricción sobre x_3 (cuando $x_1 = 0$) se impone al agregar la restricción

$$x_3 \leq x_1.$$

De manera similar, el requerimiento de que $x_4 = 0$ si $x_2 = 0$ se impone con la restricción

$$x_4 \leq x_2.$$

Por tanto, después de escribir de nuevo estas dos restricciones para que todas las variables queden en el lado izquierdo, el modelo completo de PEB es

$$\text{Maximizar} \quad Z = 9x_1 + 5x_2 + 6x_3 + 4x_4,$$

sujeta a

$$\begin{array}{rcl} 6x_1 + 3x_2 + 5x_3 + 2x_4 & \leq & 10 \\ & x_3 + x_4 & \leq 1 \\ -x_1 & + x_3 & \leq 0 \\ & -x_2 & + x_4 \leq 0 \\ & x_j & \leq 1 \\ & x_j & \geq 0 \end{array}$$

y

$$x_j \text{ es entera,} \quad \text{para } j = 1, 2, 3, 4.$$

De manera equivalente, las tres últimas líneas de este modelo se pueden sustituir por una sola restricción

$$x_j \text{ es binaria,} \quad \text{para } j = 1, 2, 3, 4.$$

Excepto por su tamaño pequeño, este ejemplo representa muchas aplicaciones reales de programación entera en las que las decisiones básicas que se toman son del tipo sí o no. Al igual que el segundo par de decisiones de este ejemplo, muchos grupos de decisiones sí o no constituyen grupos de **alternativas mutuamente excluyentes**, tales que *sólo una* decisión de ese grupo puede ser sí. Cada grupo requiere una restricción que obligue a la suma de las variables binarias correspondientes a ser igual a 1 si *exactamente* una decisión de ese grupo debe ser sí, o menor o igual a 1 (si *cualquiera* una decisión de ese grupo puede ser sí). En ocasiones, las decisiones del tipo sí o no son **decisiones contingentes**, es decir, dependen de decisiones anteriores. Por ejemplo, se dice que una decisión es *contingente* respecto a otra si se permite que sea sí *sólo si* la otra es sí. Esta situación ocurre cuando una decisión contingente implica una acción que sigue a otra y que se vuelve irrelevante, o imposible, si la otra decisión es no. La forma de la restricción que se obtiene se ilustra en la cuarta y quinta restricciones del ejemplo.

Opciones de software para resolver estos modelos

Todos los paquetes de software incluidos en el OR Courseware (Excel, LINGO/LINDO y MPL/CPLEX) contienen un algoritmo para resolver modelos de PEB (pura o mixta), así como otro para solucionar modelos generales de PE (pura o mixta) donde las variables deben ser enteras pero no binarias. Sin embargo, en razón de que las variables binarias son mucho más fáciles de manejar que las variables enteras generales, por lo común el primer algoritmo puede resolver problemas mucho más grandes que el segundo.

Cuando se utiliza Excel Solver, el procedimiento es básicamente el mismo que para programación lineal. La única diferencia surge cuando se oprime el botón de “Agregar” en el cuadro de diálogo de Solver para añadir restricciones. Además de las que se ajustan a programación lineal, se necesitan las restricciones de enteros. En el caso de variables enteras que no son binarias, éste

se logra en el cuadro de diálogo de “Agregar restricción” si se introduce el rango de variables restringidas a enteros en el lado izquierdo y después se elige “int” del menú desplegable. En el caso de variables binarias, se selecciona “bin” del menú.

En uno de los archivos de Excel para este capítulo se muestra la formulación completa en hoja de cálculo del ejemplo de la California Manufacturing Co. En la sección Worked Examples de la página en internet del libro también se incluye un **pequeño ejemplo de minimización** con dos variables restringidas a enteros. Con este caso se ilustra la formulación del modelo de PE y su solución gráfica, junto con su formulación y solución en hoja de cálculo.

Un modelo de LINGO usa la función @BIN() para especificar que las variables entre paréntesis son binarias. En el caso de una variable entera *general* (restringida a valores enteros pero no binarios), se usa la función @GIN() de la misma manera. En cualquiera de los dos casos, la función se puede anidar dentro de una instrucción @FOR para imponer estas restricciones binarias o enteras sobre un conjunto completo de variables.

En un modelo de LINDO, las restricciones binarias o enteras se insertan después de una instrucción END. Se especifica una variable X como una variable entera general con GIN X. De otra manera, para cualquier valor entero positivo de n , la instrucción GIN n especifica que las primeras n variables son enteras generales. Las variables binarias se manejan de la misma manera excepto que la palabra INTEGER se incorpora en lugar de GIN.

En el caso de un modelo de MPL, se usa la palabra clave INTEGER para designar a las variables enteras generales, mientras que BINARY indica las variables binarias. En la sección de variables de un modelo MPL, todo lo que se necesita es agregar el objetivo adecuado (INTEGER o BINARY) delante de VARIABLES para especificar que el conjunto de variables enumeradas bajo esta etiqueta es de ese tipo. Como alternativa, se puede ignorar esta especificación en la sección de variables y colocar restricciones de enteras y binarias en la sección del modelo en cualquier lugar después de las otras restricciones. En este caso, la etiqueta del conjunto de variables se convierte en sólo INTEGER o BINARY.

El complemento CPLEX de MPL incluye los últimos algoritmos para resolver modelos de PE y PEB puros o mixtos. Si elige *MIP Strategy* del submenú *CPLEX Parameters* del menú de *Options*, el usuario con experiencia puede incluso elegir entre una variedad de opciones la manera exacta de ejecutar el algoritmo para que se ajuste mejor al problema en cuestión.

Estas instrucciones para usar los paquetes de software se aclaran cuando se ve su aplicación en los ejemplos. Los archivos de Excel, LINGO/LINDO y MPL/CPLEX para este capítulo en el OR Courseware muestran cómo se aplican estas opciones con el ejemplo prototipo que se presenta en esta sección, y a otros ejemplos de PE subsecuentes.

La última parte del capítulo se centra en algoritmos de PE similares a los usados en estos paquetes. En la sección 11.6 se ilustra el algoritmo de PEB mediante la solución del ejemplo prototipo.

11.2 ALGUNAS APLICACIONES PEB

Igual que en el ejemplo de la California Manufacturing Co., con frecuencia los administradores enfrentan *decisiones de sí o no*. Por tanto, la *programación entera binaria* (PEB) se usa de manera considerable como ayuda para tomar estas decisiones.

Más adelante se introducirán varios tipos de decisiones sí o no. También se mencionan algunos ejemplos de aplicaciones reales donde se usó PEB para estudiar estas decisiones.

Cada una de estas aplicaciones se describe con detalle en un artículo de la revista *Interfaces*. Se hará hincapié en estos artículos que se incluyen en las referencias seleccionadas de aplicaciones que han sido premiadas que se citan al final del capítulo, puesto que se proporciona una liga a dichos artículos en el sitio en internet de este libro. En el caso de los demás artículos, se mencionará el número específico de la revista *Interfaces* en el que aparece el artículo.

Análisis de inversión

En ocasiones se usa programación lineal para tomar decisiones de presupuesto de capital acerca de cuánto invertir en diferentes proyectos. Sin embargo, como lo hace evidente el ejemplo de la California Manufacturing, algunas decisiones de presupuestos no se refieren a *cuánto* invertir sino

al hecho de *si debe* invertirse una cantidad fija. En especial, las cuatro decisiones del ejemplo eran si invertir una cantidad fija que es requerida para construir cierto tipo de instalación (fábrica o almacén) en determinado lugar (Los Ángeles o San Francisco).

La administración suele encontrarse en situaciones de decisión sobre si hacer una inversión fija (en las cuales la cantidad de capital se establece desde antes. ¿Debe adquirirse una subsidiaria en proceso de cierre de otra compañía? ¿Debe preferirse a cierto proveedor de materia prima? ¿Debe agregarse una nueva línea de producción para fabricar cierto subensamble en lugar de continuar con determinado proveedor?

En general, las decisiones de presupuestos de capital sobre inversiones fijas son decisiones sí o no del tipo siguiente.

Para cada decisión sí o no:

¿Debe realizarse cierta inversión fija?

$$\text{Variable de decisión} = \begin{cases} 1 & \text{si sí} \\ 0 & \text{si no.} \end{cases}$$

El número de julio-agosto de 1990 de *Interfaces* describe cómo *Turkish Petroleum Refineries Corporation* utiliza el enfoque PEB para analizar inversiones de capital del orden de decenas de millones de dólares para ampliar la capacidad de una refinería y ahorrar energía.

Un ejemplo distinto que todavía pertenece a esta categoría se describe en la referencia seleccionada A7. El enorme estudio de IO que se realizó para la *South African National Defense Force* para actualizar su capacidad con un presupuesto más pequeño. Las “inversiones” bajo consideración en este caso fueron los costos de adquisición y gastos de operación requeridos para proporcionar los tipos específicos de capacidad militar. Se formuló un modelo mixto de PEB con el objetivo de elegir las capacidades específicas para maximizar la eficacia global de la fuerza militar y satisfacer la restricción de presupuesto. El modelo tiene más de 16 mil variables —que incluyen 256 variables binarias— y más de 5 mil restricciones funcionales. La optimización acerca del tamaño y forma de la fuerza militar proporcionó ahorros de más de 1 100 millones de dólares anuales lo mismo que beneficios no monetarios. El efecto de este estudio hizo que ganara en 1996 el Franz Edelman Award por logros en Ciencias de la administración.

En una aplicación militar que de alguna forma es similar, la Comandancia Espacial de la Fuerza Aérea de Estados Unidos gasta muchos miles de millones de dólares cada año para adquirir y desarrollar vehículos y sistemas espaciales. El número de julio-agosto de 2003 de *Interfaces* describe la forma como la Comandancia Espacial emplea programación entera para optimizar estas inversiones a largo plazo en un horizonte de planeación de 24 años.

En la referencia seleccionada A3 se presenta otra aplicación premiada de un modelo de PEB mixto al análisis de inversiones. Este modelo particular fue utilizado por la compañía de inversiones *Grantham, Mayo, Van Otterloo and Company* para construir numerosos portafolios manejados de manera cuantitativa que representan más de 8 mil millones de dólares en títulos. En cada caso, se ha construido un portafolio parecido —en términos de sector y exposición de seguridad— a un portafolio objetivo pero con un número mucho más pequeño y manejable de acciones diferentes. Se utiliza una variable binaria para representar cada decisión del tipo sí o no para determinar si una acción particular se debe incluir en el portafolio; después, una variable continua representa la cantidad que se debe invertir en esa acción. Dado un portafolio que necesita ser rebalanceado, es deseable reducir los costos de transacción a través de minimización del número de operaciones necesarias para obtener el portafolio final, por lo que también es necesario incluir variables binarias para representar las decisiones del tipo sí o no para determinar si es necesario llevar a cabo las transacciones para cambiar las cantidades de las acciones que se posee. La inclusión de esta consideración en el modelo ha reducido el costo anual de transacciones de portafolios en al menos 4 millones de dólares.

Elección del sitio

En la economía globalizada, muchas corporaciones instalan nuevas plantas en diversas regiones del planeta para aprovechar la mano de obra más barata y otras ventajas. Antes de elegir la ubicación de esas instalaciones, deben analizarse y compararse muchos sitios potenciales. (El ejemplo de la California Manufacturing Co., tenía sólo dos sitios potenciales para cada una de dos tipos de instalaciones.) Cada ubicación potencial involucra una decisión sí o no del siguiente tipo.

Para cada decisión sí o no:

¿Debe elegirse cierto lugar para la ubicación de cierta instalación nueva?

$$\text{Variable de decisión} = \begin{cases} 1 & \text{si sí} \\ 0 & \text{si no.} \end{cases}$$

En muchos casos, el objetivo es seleccionar los sitios para minimizar el costo total de las nuevas instalaciones que proporcionarán la producción que se requiere.

Como se describe en la referencia seleccionada A11, AT&T utilizó un modelo de PEB para ayudar a docenas de clientes a seleccionar sitios para sus centros de telemercadeo. El modelo minimiza los costos de mano de obra, comunicaciones y bienes raíces a la vez que proporciona el nivel deseado de cobertura de los centros. En sólo un año (1988), este enfoque permitió a 46 clientes de AT&T tomar sus decisiones sí o no sobre las ubicaciones con presteza y confianza, al mismo tiempo que comprometieron \$375 millones anuales en servicios de red y \$31 millones en venta de equipo de AT&T.

A continuación describiremos un tipo importante de problemas en los que la selección del sitio tiene un papel esencial.

Diseño de una red de producción y distribución

Los fabricantes de hoy se enfrentan a una gran presión competitiva para lograr que sus productos lleguen al mercado con mayor rapidez, así como para reducir sus costos de producción y distribución. Por tanto, cualquier corporación que distribuye sus productos en un área geográfica grande —o incluso en todo el mundo— debe prestar una atención continua al diseño de sus productos y a la red de distribución.

Este diseño incluye investigar los siguientes tipos de decisiones sí o no.

¿Debe cierta planta permanecer abierta?

¿Debe seleccionarse cierto sitio para una nueva planta?

¿Debe cierto centro de distribución permanecer abierto?

¿Debe cierto sitio elegirse para instalar un nuevo centro de distribución?

Si cada área de mercado debe recibir servicio de un solo centro, entonces también es necesario tomar otro tipo de decisiones sí o no sobre cada combinación de área de mercado y centro de distribución.

¿Debe cierto centro de distribución asignarse a cierta área de mercado?

Por cada decisión sí o no de cualquiera de estos tipos:

$$\text{Variable de decisión} = \begin{cases} 1 & \text{si sí} \\ 0 & \text{si no.} \end{cases}$$

Ault Foods Limited (número de julio-agosto de 1994 de *Interfaces*) usó este enfoque para diseñar su centro de producción y distribución. La administración consideró 10 lugares para plantas, 13 lugares para centros de distribución y 48 áreas de mercado. Esta aplicación de PEB fue responsable de que la empresa pudiese ahorrar 200 mil dólares anuales.

Digital Equipment Corporation—este caso se describe en el número de enero-febrero de 1995 de *Interfaces*— es otro ejemplo de una aplicación de este tipo. En ese tiempo, esta gran corporación multinacional atendía 250 mil instalaciones de sus clientes, y la mitad de sus 14 mil millones de ingresos anuales provenían de 81 países fuera de Estados Unidos. Por tanto, esta aplicación consideraba la reestructuración de toda la *cadena de abastecimiento global* de la empresa, que consistía en proveedores, plantas, centros de distribución, sitios potenciales y áreas de mercado en todo el mundo. La reestructuración provocó reducciones de costos anuales por 500 millones en producción y 300 millones en logística, además de una reducción de más de 400 millones en activos de capital.

Despacho de envíos

Una vez que la red de producción y distribución haya sido diseñada y puesta en operación, deben tomarse decisiones operativas diarias acerca de cómo realizar los envíos. Algunas de estas decisiones también son de sí o no.

Por ejemplo, suponga que se usan camiones para transportar los envíos y que cada camión suele hacer entregas a varios clientes durante cada viaje. En consecuencia, es necesario elegir una ruta (secuencia de clientes) para cada camión, de manera que cada candidato para la ruta conduce a la siguiente decisión de sí o no.

¿Debe cierta ruta seleccionarse para uno de los camiones?

$$\text{Variable de decisión} = \begin{cases} 1 & \text{si sí} \\ 0 & \text{si no.} \end{cases}$$

El objetivo es seleccionar las rutas que minimizan el costo total de realizar los envíos.

También se pueden tomar en cuenta algunas complicaciones. Por ejemplo, si se dispone de camiones de varios tamaños, cada candidato para la elección debe incluir cierta ruta y cierto tamaño de camión. De igual manera, si el tiempo es un factor, se puede especificar el momento de salida como parte de la decisión de sí o no. Con ambos factores, cada decisión sí o no tendría la forma que se muestra a continuación.

Deben seleccionarse los siguientes elementos de manera simultánea para una entrega:

1. Cierta ruta,
2. Cierta tamaño de camión y
3. Cierta momento de salida

$$\text{Variable de decisión} = \begin{cases} 1 & \text{si sí} \\ 0 & \text{si no.} \end{cases}$$

Por ejemplo, *Sears, Roebuck and Company* —consulte el número de enero-febrero de 1999 de *Interfaces*— logró ahorros anuales por más de 42 millones de dólares mediante el empleo de un sistema de vehículos-rutas-y-programación basado en PEB y otro de información geográfica para realizar su servicio de entregas a domicilio de manera más eficiente.

Programación de actividades interrelacionadas

En la vida diaria, todas las personas programan actividades interrelacionadas, aun si sólo implican organizar el inicio de las tareas de la escuela. Los administradores también deben programar varios tipos de actividades interrelacionadas. ¿Cuándo se debe iniciar la producción de las nuevas órdenes? ¿Cuándo deben comercializarse los nuevos productos? ¿Cuándo deben realizarse las inversiones de capital para ampliar la capacidad de producción?

Para cualquier actividad como éstas, la decisión acerca de cuándo comenzar se puede expresar en términos de una serie de decisiones sí o no, una de ellas para cada momento posible de inicio, como se muestra en seguida.

¿Debe cierta actividad comenzar en un momento dado?

$$\text{Variable de decisión} = \begin{cases} 1 & \text{si sí} \\ 0 & \text{si no.} \end{cases}$$

Como una actividad en particular puede comenzar sólo en un momento, la elección de varios momentos proporciona un grupo de *alternativas mutuamente excluyentes*, de manera que la variable de decisión para uno solo de ellos puede tener un valor de 1.

Por ejemplo, considere la siguiente aplicación que ocurrió en *China* —número de enero-febrero de 1995 de *Interfaces*—. El país se enfrentaba a inversiones de al menos 240 mil millones de dólares en un horizonte de 15 años para cumplir con las necesidades de energía de su economía creciente. Para cubrir los faltantes de carbón y electricidad era necesario desarrollar una infraes-

Recuadro de aplicación

Las líneas aéreas comerciales deben resolver dos problemas complejos de programación de vuelos con el fin de garantizar que todos los vuelos que se programen tengan una tripulación disponible. Uno de ellos, llamado *problema de la planeación de viajes ocupados*, involucra la elaboración de secuencias de vuelo con periodos de descanso entre ellas que comprometerán los viajes ocupados por muchos días para las tripulaciones individuales. El segundo, que se llama *problema de las listas de turnos*, involucra la asignación de dichos viajes ocupados a cada uno de los miembros de la tripulación. La gerencia trata de encontrar soluciones de costo mínimo y máxima productividad a dichos problemas y así como satisfacer los acuerdos laborales y considerar las preferencias de los miembros de la tripulación.

Una gran cantidad de líneas aéreas alrededor del mundo han logrado ahorros significativos en años recientes mediante el uso de *modelos PEB* para obtener soluciones óptimas a dichos problemas. Una de estas aerolíneas es **Air New Zealand**, que es la línea aérea nacional e internacional más grande con base en este país. Emplea a más de 2 000 miembros de la

tripulación y opera vuelos a Australia, Asia, Norteamérica y Europa, así como entre los centros más importantes de Nueva Zelanda.

Por lo general, los modelos PEB usados por la Air New Zealand cuentan con *cientos de restricciones funcionales y muchos miles de variables binarias*, en los que se utilizan técnicas avanzadas para resolver dichos modelos. Una estimación conservadora del ahorro que resulta del uso de estos modelos es de alrededor de **6.7 millones de dólares al año**, lo cual representa 11% de la ganancia operativa de la compañía en un año reciente. Asimismo, se presentaron muchos beneficios intangibles en los que se incluye una rápida implantación, la eficiente reprogramación de cambios de última hora y una mejora del servicio a los pasajeros.

Fuente: E. R. Butchers, P. R. Day, A. P. Goldie, S. Miller, J. A. Meyer, D. M. Ryan, A. C. Scott y C. A. Wallace: "Optimized Crew Scheduling at Air New Zealand", en *Interfaces*, **31**(1): 30-56, enero-febrero de 2001. (En el sitio en internet de este libro, www.mhhe.com/hillier, se proporciona una liga hacia este artículo.)

estructura para transportar carbón y transmitir energía, la construcción de nuevas presas y plantas para generar energía térmica, hidráulica y nuclear. Por tanto, la Comisión de Planeación del Gobierno de China y el Banco Mundial colaboraron para elaborar un gran modelo de PEB que debía guiar las decisiones acerca de qué proyectos aprobar y cuándo iniciarlos en un periodo de 15 años, para minimizar el costo descontado total. Se estima que esta aplicación de IO ha ahorrado a China alrededor de 6 400 millones de dólares en ese periodo.

Aplicaciones a líneas aéreas

La industria de líneas aéreas es un usuario constante de la IO en todas sus operaciones. Cientos de profesionales de la Investigación de Operaciones trabajan en esta área. Por lo general, las compañías aéreas más importantes tienen un departamento propio que trabaja en aplicaciones de IO. Además, hay algunas firmas de consultoría prominentes que se enfocan sólo en los problemas de compañías relacionadas con el transporte, en especial las líneas aéreas. Mencionaremos dos de las aplicaciones que se basan en PEB de manera directa.

Uno es el *problema de asignación de flota*. Dados varios modelos de aviones disponibles, el problema es asignar un tipo específico a cada tipo de trayecto de manera que se maximice la ganancia total si se cumple con lo programado. El trueque básico estriba en que si la línea aérea usa un avión demasiado pequeño en un trayecto dado, dejará clientes potenciales en tierra, mientras que si asigna uno demasiado grande, tendrá un gasto mucho mayor y volará con asientos vacíos.

Para cada combinación de tipo de avión y trayecto, se tiene la siguiente decisión sí o no.

¿Debe asignarse cierto tipo de avión a un trayecto en particular?

$$\text{Variable de decisión} = \begin{cases} 1 & \text{si sí} \\ 0 & \text{si no.} \end{cases}$$

Delta Air Lines realiza más de 2 500 vuelos nacionales todos los días, y emplea alrededor de 450 aviones de varios tipos. Como se describió en la referencia seleccionada A12, la empresa utiliza un modelo de programación lineal enorme —cerca de 40 mil restricciones funcionales, 20 mil variables binarias y 40 mil variables enteras— para resolver su problema de asignación de la flota cada vez que se requiere un cambio. Esta aplicación ahorra a Delta cerca de \$100 millones anuales.

Una aplicación similar es el *problema de programación de tripulaciones*. En éste, en lugar de asignar tipos de aviones a rutas de vuelo, se asignan secuencias de rutas de vuelo a las tripulaciones

de pilotos y asistentes de vuelo. De esta forma, por cada secuencia factible de vuelos que sale de una base y regresa a la misma base, debe tomarse la siguiente decisión sí o no.

¿Debe cierta secuencia de rutas de vuelo asignarse a una tripulación?

$$\text{Variable de decisión} = \begin{cases} 1 & \text{si sí} \\ 0 & \text{si no.} \end{cases}$$

El objetivo es minimizar el costo total de proporcionar tripulaciones que cubran cada ruta según la programación.

American Airlines —este caso aparece en los números de julio-agosto de 1989 y enero-febrero de 1991 de *Interfaces*— logra ahorros anuales de más de \$20 millones con el uso de PEB para resolver sus problemas de programación mensual de tripulaciones. Este enfoque también es muy usado por compañías aéreas que tienen sus oficinas centrales fuera de Estados Unidos. Por ejemplo, *Air New Zealand* —número de enero-febrero de 2001 de *Interfaces*— ahorra aproximadamente 6.7 millones de dólares al año debido al uso de PEB para optimizar la programación de tripulaciones, como se describió en el recuadro de aplicación de la página anterior.

Al final de la sección 11.4 se presentará un ejemplo de formulación completa de este tipo.

A las compañías aéreas se les presenta un problema similar cuando los programas de sus tripulaciones deben ser revisados de manera ocasional y con rapidez ante el acacamiento de retrasos o cancelaciones de vuelos debidos a inclemencias del clima, problemas mecánicos en las naves o no disponibilidad de las tripulaciones. Como se describió en la Viñeta de aplicación de la sección 2.2 (así como en la referencia seleccionada A14), *Continental Airlines* logró ahorros de 40 millones de dólares en el primer año de uso de un sistema de apoyo a las decisiones basado en PEB para optimizar la *reasignación* de tripulaciones cuando ocurren tales emergencias. (En el año 2002, Continental Airlines ganó el primer lugar de los Franz Edelman Awards por los logros en ciencias de la administración por esta aplicación innovadora.)

Muchos de los problemas que enfrentan las compañías aéreas también surgen en otros segmentos de la industria del transporte. Por tanto, algunas de las aplicaciones de IO a los aviones se han extendido a estas otras áreas, incluso a los viajes en tren. Por ejemplo, tres de los ganadores al primer lugar de los Franz Edelman Award por logros en Ciencias de la administración en años recientes han sido por aplicaciones a la empresa nacional de ferrocarriles de Francia, la Canadian Pacific Railway y la Netherlands Railways que lograron beneficios financieros sorprendentes. (Vea los números de enero-febrero de 1998 y 2004 de *Interfaces*.) La referencia seleccionada A1 también describe la forma en que Netherlands Railways (NS Reizigers) ahorra en la actualidad 4.8 millones de dólares al año mediante el uso de BIP para optimizar la programación de su tripulación.

11.3 USOS INNOVADORES DE VARIABLES BINARIAS EN LA FORMULACIÓN DE MODELOS

Se han presentado varios ejemplos en los que las *decisiones básicas* del problema son del tipo *sí o no*, por lo que se introdujeron *variables binarias* para representarlas. Ahora se verán algunas otras formas en las que las variables binarias pueden ser útiles. En particular, se verá que estas variables a veces permiten tomar un problema cuya formulación no se puede manejar y *reformularlo* como un problema de PE pura o mixta.

Este tipo de situación surge cuando la formulación original del problema se ajusta a un problema de programación entera o a uno de programación lineal, *excepto* por ciertas diferencias menores que incluyen relaciones combinatorias en el modelo. Cuando se expresan estas relaciones en términos de preguntas cuya respuesta debe ser sí o no, se pueden introducir al modelo **variables binarias auxiliares** para representar estas decisiones sí o no. (En lugar de ser una variable de decisión del problema original en consideración, una variable binaria *auxiliar* es una variable binaria que se introduce en el modelo del problema con la finalidad de ayudar a formular el modelo como un modelo BIP puro o mezclado.) Al introducir estas variables, el problema se reduce a uno de PEM (o PE pura si todas las variables originales también están restringidas a valores enteros).

En seguida se presentan algunos casos que se pueden formular con este enfoque, donde las x_j denotan las variables *originales* del problema —pueden ser variables continuas o enteras—, y las y_i denotan las variables *auxiliares* que se introdujeron para realizar la nueva formulación.

Restricciones de tipo una u otra

Considere el importante caso en el que se debe elegir entre dos restricciones, de manera que *sólo una* (cualquiera de las dos) se tiene que cumplir (mientras que la otra puede cumplirse pero no es imprescindible que lo haga). Por ejemplo, puede existir la opción de usar uno de dos tipos de recursos para cierto propósito, de manera que sólo es necesario que una de las restricciones de disponibilidad de estos recursos se cumpla en forma matemática. Para ilustrar este tipo de situaciones, suponga que uno de los requerimientos en el problema completo es que

$$\begin{array}{ll} \text{se cumple} & 3x_1 + 2x_2 \leq 18 \\ \text{o} & x_1 + 4x_2 \leq 16, \end{array}$$

es decir, al menos una de estas dos desigualdades debe cumplirse pero no necesariamente las dos. Este requerimiento debe reformularse para que el problema se ajuste al formato de programación lineal, en el que se deben cumplir *todas* las restricciones especificadas. Si M es un número positivo muy grande, la expresión anterior se puede reescribir como

$$\begin{array}{ll} \text{se cumple} & \begin{array}{l} 3x_1 + 2x_2 \leq 18 \\ x_1 + 4x_2 \leq 16 + M \end{array} \\ \text{o} & \begin{array}{l} 3x_1 + 2x_2 \leq 18 + M \\ x_1 + 4x_2 \leq 16. \end{array} \end{array}$$

La clave es que al agregar M al lado derecho de una restricción de este tipo se obtiene el efecto de eliminarla, puesto que cualquier solución que satisfaga las otras restricciones del problema cumplirá ésta de manera automática. (Esta formulación supone que el conjunto de soluciones factibles del problema completo es un conjunto acotado y que M es lo suficientemente grande como para no eliminar ninguna de las soluciones factibles.) Esta formulación es equivalente al conjunto de restricciones

$$\begin{array}{l} 3x_1 + 2x_2 \leq 18 + My \\ x_1 + 4x_2 \leq 16 + M(1 - y). \end{array}$$

Debido a que la *variable auxiliar* y debe ser 0 o 1, esta formulación garantiza que una de las restricciones originales se cumple mientras que la otra es, de hecho, eliminada. Este nuevo conjunto de restricciones se podría añadir a las otras restricciones del modelo completo para obtener un problema de PE pura o mixta, lo cual depende de las x_j sean variables enteras o continuas.

Este enfoque se relaciona en forma directa con el análisis anterior sobre cómo expresar las relaciones combinatorias en términos de preguntas que deben responderse con sí o no. La relación combinatoria a la que se hace referencia concierne a la combinación de *otras* restricciones del modelo con la *primera* de las dos restricciones *alternas* y después con la *segunda*. ¿Cuál de estas dos combinaciones de restricciones es *mejor*, es decir, que se puede lograr en cuanto al valor de la función objetivo? Si esta pregunta se hace en términos de sí o no, en realidad es necesario formular dos preguntas complementarias:

1. ¿Debe elegirse $x_1 + 4x_2 \leq 16$ como la restricción que debe cumplirse?
2. ¿Debe elegirse $3x_1 + 2x_2 \leq 18$ como la restricción que debe cumplirse?

Como sólo una de estas dos preguntas tendrá una respuesta afirmativa, los respectivos términos binarios y y $1 - y$ representan las decisiones sí o no. En consecuencia, $y = 1$ si la respuesta a la primera pregunta es sí (y no a la segunda), mientras que $1 - y = 1$ (esto es $y = 0$) si la respuesta a la segunda pregunta es sí (y no a la primera). Como de manera automática $y + 1 - y = 1$ (un sí), no hay necesidad de agregar otra restricción para forzar a estas dos decisiones a que sean mutuamente excluyentes. (Si en lugar de este procedimiento se usan las variables binarias separadas y_1 y y_2 para representar estas decisiones sí o no, se necesitará una restricción adicional $y_1 + y_2 = 1$ para hacerlas mutuamente excluyentes.)

A continuación se ofrece una presentación formal de este enfoque para un caso más general.

Deben cumplirse K de N restricciones

Considere la situación en la que el modelo completo incluye un conjunto de N restricciones posibles entre las que sólo K de ellas se *deben* cumplir. (Suponga que $K < N$.) Parte del proceso de optimización es elegir la *combinación* de K restricciones que permita que la función objetivo adquiera el mejor valor posible. En realidad, las $N - K$ restricciones que *no* se eligen son eliminadas del problema, aun cuando por coincidencia las soluciones factibles puedan satisfacer algunas de ellas.

Este caso es una generalización directa del anterior que tenía $K = 1$ y $N = 2$. Denote las N restricciones posibles por

$$\begin{aligned} f_1(x_1, x_2, \dots, x_n) &\leq d_1 \\ f_2(x_1, x_2, \dots, x_n) &\leq d_2 \\ &\vdots \\ f_N(x_1, x_2, \dots, x_n) &\leq d_N. \end{aligned}$$

Después, al aplicar la misma lógica que en el caso anterior, se encuentra que una formulación equivalente del requerimiento de que K de estas restricciones se *deben* cumplir es

$$\begin{aligned} f_1(x_1, x_2, \dots, x_n) &\leq d_1 + My_1 \\ f_2(x_1, x_2, \dots, x_n) &\leq d_2 + My_2 \\ &\vdots \\ f_N(x_1, x_2, \dots, x_n) &\leq d_N + My_N \\ \sum_{i=1}^N y_i &= N - K, \end{aligned}$$

y

$$y_i \text{ es binaria, para } i = 1, 2, \dots, N,$$

donde M es un número positivo muy grande. Para cada variable binaria y_i ($i = 1, 2, \dots, N$), observe que $y_i = 0$ hace que $My_i = 0$, lo que reduce la nueva restricción a la restricción original i . Por otro lado, $y_i = 1$ hace que $(d_i + My_i)$ sea tan grande que —si otra vez se supone una región factible acotada— cualquier solución que satisfaga las otras nuevas restricciones satisface de manera automática la nueva restricción i , lo cual tiene el efecto de eliminar la restricción original i . Por tanto, en razón de que las restricciones sobre las variables y_i garantizan que K de estas variables serán iguales a 0 y las restantes serán iguales a 1, K de las restricciones originales quedarán sin cambio y $(N - K)$ de ellas serán eliminadas. La elección de *cuáles* K de estas restricciones deben conservarse se hace mediante la aplicación del algoritmo apropiado al problema completo para encontrar una solución óptima para *todas* las variables de manera simultánea.

Funciones con N valores posibles

Considere la situación en la que se requiere que una función dada tome cualquiera de N valores dados. Denote este requisito por

$$f(x_1, x_2, \dots, x_n) = d_1 \quad \text{o} \quad d_2, \dots, \quad \text{o} \quad d_N.$$

Un caso especial es aquel en el que esta función es

$$f(x_1, x_2, \dots, x_n) = \sum_{j=1}^n a_j x_j,$$

como en el lado izquierdo de una restricción de programación lineal. Otro caso especial es en el que $f(x_1, x_2, \dots, x_n) = x_j$ para un valor dado de j , de manera que el requerimiento en realidad es que x_j debe tomar cualquiera de los N valores dados.

La formulación equivalente de PE de este requerimiento es la siguiente:

$$f(x_1, x_2, \dots, x_n) = \sum_{i=1}^N d_i y_i$$

$$\sum_{i=1}^N y_i = 1$$

y

y_i es binaria, para $i = 1, 2, \dots, N$,

con lo que este nuevo conjunto de restricciones sustituye el requerimiento que se hizo al establecer el problema completo. Este conjunto proporciona una formulación *equivalente* puesto que exactamente una de las y_i debe ser igual a 1 y las otras deben ser iguales a 0, así que se escoge una d_i como el valor de la función. En este caso existen N preguntas con respuesta sí o no, que son: ¿debe ser d_i el valor escogido ($i = 1, 2, \dots, N$)? Como la y_i respectiva representa estas *decisiones sí o no*, la segunda restricción las hace *alternativas mutuamente excluyentes*.

Para ilustrar cómo puede surgir un caso de este tipo, reconsidere el problema de la Wyndor Glass Co., que se presentó en la sección 3.1. En la actualidad no se emplean 18 horas de producción a la semana en la planta 3 y están disponibles para los dos nuevos productos o para otros que muy pronto estarán listos para producción. Con el fin de dejar cualquier capacidad restante en forma de bloques utilizables para estos productos, la administración desea imponer la restricción de que el tiempo de producción que se empleará en los dos nuevos productos actuales sea 6 o 12 o 18 horas semanales. Entonces, la tercera restricción del modelo original ($3x_1 + 2x_2 \leq 18$) se convierte en

$$3x_1 + 2x_2 = 6 \quad \text{o} \quad 12 \quad \text{o} \quad 18.$$

En la notación anterior, $N = 3$ con $d_1 = 6$, $d_2 = 12$ y $d_3 = 18$. En consecuencia, el nuevo requerimiento de la administración se debe formular de la siguiente manera:

$$3x_1 + 2x_2 = 6y_1 + 12y_2 + 18y_3$$

$$y_1 + y_2 + y_3 = 1$$

y

y_1, y_2, y_3 son binarias.

El modelo completo de esta nueva versión del problema consiste, entonces, en el modelo original (vea la sección 3.1), más este nuevo conjunto de restricciones que reemplaza a la tercera restricción original. Esta sustitución conduce a una formulación de PEM muy clara.

Problema de costo fijo

Es bastante común incurrir en un cargo de preparación o costo fijo cuando se emprende una actividad. Por ejemplo, un cargo así ocurre cuando se inicia una corrida de producción de un lote pequeño de un producto específico y deben prepararse las instalaciones fabriles que se requieren. En tales casos, el costo total de la actividad es la suma de un costo variable relacionado con el nivel de la actividad y el costo fijo en el que se incurre para iniciarla. Con frecuencia, el costo variable estará muy cerca de ser proporcional al nivel de la actividad. Si lo es, el *costo total* de la actividad (por ejemplo, j) se puede representar mediante una función de la forma

$$f_j(x_j) = \begin{cases} k_j + c_j x_j & \text{si } x_j > 0 \\ 0 & \text{si } x_j = 0, \end{cases}$$

donde x_j denota el nivel de la actividad j ($x_j \geq 0$), k_j denota el costo fijo y c_j el costo de cada unidad incremental. Si no fuera por el costo fijo k_j esta estructura de costo sugeriría la posibilidad de una formulación de *programación lineal* para determinar los niveles óptimos de las actividades competitivas. Por fortuna, aun si se incluyen las k_j , se puede usar programación entera.

Para formular el modelo completo, suponga que existen n actividades, cada una con la estructura de costo anterior —con $k_j \geq 0$ en todos los casos y $k_j > 0$ para alguna $j = 1, 2, \dots, n$ —, y que el problema es

$$\text{Minimizar} \quad Z = f_1(x_1) + f_2(x_2) + \dots + f_n(x_n),$$

sujeta a

las restricciones de programación lineal dadas.

Para convertir este problema al formato de programación entera mixta, se comienza por proponer las preguntas que deban responderse con sí o no; por ejemplo, para cada $j = 1, 2, \dots, n$, ¿debe emprenderse la actividad j ($x_j > 0$)? Después, cada una de estas decisiones sí o no se representa por una *variable binaria* y_j , de manera que

$$Z = \sum_{j=1}^n (c_j x_j + k_j y_j),$$

donde

$$y_j = \begin{cases} 1 & \text{si } x_j > 0 \\ 0 & \text{si } x_j = 0. \end{cases}$$

Por tanto, se puede pensar que las y_j son *decisiones contingentes* parecidas (pero no idénticas) a las que se consideran en la sección 11.1. Si M es un número positivo muy grande que excede el máximo valor factible de cualquiera de las x_j ($j = 1, 2, \dots, n$), las restricciones

$$x_j \leq M y_j \quad \text{para } j = 1, 2, \dots, n$$

asegurarán que $y_j = 1$ y no cero, siempre que $x_j > 0$. La única dificultad que queda por salvar es que estas restricciones dejan a las y_j en libertad de tomar el valor 0 o 1 cuando $x_j = 0$. Por fortuna, esto también se resuelve en forma automática por la naturaleza de la función objetivo. El caso en el que $k_j = 0$ se puede ignorar puesto que y_j se puede eliminar de la formulación. Entonces, sólo se toma en cuenta el otro caso, es decir, cuando $k_j > 0$. Si $x_j = 0$, de manera que las restricciones permiten elegir entre $y_j = 0$ y $y_j = 1$, $y_j = 0$ debe conducir a un valor menor de Z que $y_j = 1$. De esta forma, como el objetivo es minimizar Z , un algoritmo que conduzca a una solución óptima siempre elegirá $y_j = 0$ si $x_j = 0$.

Para resumir, la formulación de PEM del problema de costo fijo es

$$\text{Minimizar} \quad Z = \sum_{j=1}^n (c_j x_j + k_j y_j),$$

sujeta

las restricciones originales, más

$$x_j - M y_j \leq 0$$

y

$$y_j \text{ es binaria,} \quad \text{para } j = 1, 2, \dots, n.$$

Si las x_j también estuvieran restringidas a valores enteros, éste sería un problema de PE *pura*.

Para ilustrar este enfoque, vea de nuevo el problema de la contaminación ambiental que enfrenta la Nori & Leets Co., que se presentó en la sección 3.4. El primer método de reducción que se consideró —aumentar la altura de las chimeneas— en realidad significaría un enorme *costo fijo* para poder realizar *cualquier* aumento, además de un costo variable que en esencia sería proporcional al aumento. Después de una conversión al costo anual equivalente que se utilizó en la formulación, este cargo fijo sería de 2 millones de dólares por cada uno de los altos hornos y los hornos de corazón abierto, mientras que los costos variables son los mismos que se identificaron en la tabla 3.14. Por tanto, en la notación anterior, $k_1 = 2$, $k_2 = 2$, $c_1 = 8$ y $c_2 = 10$, donde la función objetivo se expresa en unidades de *millones* de dólares. Como los otros métodos de reducción no

implican cargos fijos, $k_j = 0$ para $j = 3, 4, 5, 6$. En consecuencia, la nueva formulación de PEM de este problema es

$$\text{Minimizar } Z = 8x_1 + 10x_2 + 7x_3 + 6x_4 + 11x_5 + 9x_6 + 2y_1 + 2y_2,$$

sujeta a

las restricciones dadas en la sección 3.4, más

$$x_1 - My_1 \leq 0,$$

$$x_2 - My_2 \leq 0,$$

y

y_1, y_2 son binarias.

Representación binaria de variables enteras en general

Suponga que se tiene un problema de PE pura en el que la mayoría de las variables son *binarias*, pero la presencia de algunas variables enteras *generales* evita que se pueda usar el eficiente algoritmo de PEB ahora disponible. Una manera de salvar esta dificultad es usar la *representación binaria* de cada una de estas variables enteras generales. En particular, si las cotas de una variable entera x son

$$0 \leq x \leq u$$

y si N está definido como un entero tal que

$$2^N \leq u < 2^{N+1},$$

entonces la **representación binaria** de x es

$$x = \sum_{i=0}^N 2^i y_i,$$

donde las variables y_i son variables binarias (auxiliares). Si se sustituye esta representación binaria por cada variable entera general —con un conjunto diferente de variables binarias auxiliares de cada una—, el problema completo se reduce a un modelo de PEB.

Por ejemplo, suponga que un problema de PE tiene sólo dos variables enteras generales, x_1 y x_2 y muchas variables binarias. Suponga también que el problema tiene restricciones de no negatividad para ambas variables y que las restricciones funcionales incluyen

$$\begin{aligned} x_1 &\leq 5 \\ 2x_1 + 3x_2 &\leq 30. \end{aligned}$$

Estas restricciones implican que $u = 5$ para x_1 y $u = 10$ para x_2 , de manera que la definición anterior de N da $N = 2$ para x_1 —puesto que $2^2 \leq 5 < 2^3$ — y $N = 3$ para x_2 (porque $2^3 \leq 10 < 2^4$). Entonces, las representaciones binarias de estas variables son

$$\begin{aligned} x_1 &= y_0 + 2y_1 + 4y_2 \\ x_2 &= y_3 + 2y_4 + 4y_5 + 8y_6. \end{aligned}$$

Después de sustituir estas expresiones por las variables respectivas en todas las restricciones funcionales y en la función objetivo, las dos restricciones funcionales establecidas son:

$$\begin{aligned} y_0 + 2y_1 + 4y_2 &\leq 5 \\ 2y_0 + 4y_1 + 8y_2 + 3y_3 + 6y_4 + 12y_5 + 24y_6 &\leq 30. \end{aligned}$$

Observe que cada valor factible de x_1 corresponde a uno de los valores factibles del vector (y_0, y_1, y_2) , lo cual también ocurre con x_2 y (y_3, y_4, y_5, y_6) . Por ejemplo, $x_1 = 3$ corresponde a $(y_0, y_1, y_2) = (1, 1, 0)$ y $x_2 = 5$ corresponde a $(y_3, y_4, y_5, y_6) = (1, 0, 1, 0)$.

Si se trata de un problema de PE en el que *todas* las variables son enteras generales (acotadas), es posible usar esta misma técnica para reducirlo a un modelo de PEB. Sin embargo, esto casi nunca

es aconsejable debido a la explosión del número de variables involucradas. En general, aplicar un buen algoritmo de PE al modelo original será más eficiente que aplicar un buen algoritmo de PEB al modelo mucho más grande de PEB.¹

En términos generales, con *todas* las posibilidades de formulación con variables binarias auxiliares que se presentan en esta sección, es necesario hacer notar una precaución que debe tenerse. Algunas veces, este enfoque requiere que se agregue un número relativamente grande de variables, lo que puede hacer que el modelo se vuelva *no factible en cuanto a los cálculos*. (En la sección 11.5 se proporcionan algunas perspectivas sobre los tamaños de los problemas de PE que se pueden resolver.)

■ 11.4 ALGUNOS EJEMPLOS DE FORMULACIÓN

A continuación se presentará una serie de ejemplos que ilustran varias técnicas de formulación con variables binarias, entre ellos los que se estudiaron en las dos secciones anteriores. Por claridad, estos ejemplos se han mantenido muy pequeños. (En el sitio en internet de este libro, dentro de la sección de Worked Examples, se incluye un **ejemplo de formulación más grande** con docenas de variables binarias y restricciones). En las aplicaciones reales, estas formulaciones son sólo una pequeña parte de un modelo mucho más amplio.

EJEMPLO 1 Elecciones cuando las variables de decisión son continuas

La división de investigación y desarrollo de la GOOD PRODUCTS COMPANY ha desarrollado tres nuevos productos posibles. Sin embargo, para evitar una diversificación excesiva de la línea de productos de la compañía, la administración ha impuesto las siguientes limitaciones.

Restricción 1: De los tres nuevos productos posibles, deben escogerse, *como máximo*, sólo *dos* de ellos.

Se dispone de dos plantas que pueden fabricar los productos elegidos. Por razones administrativas, la administración impuso una segunda restricción a este respecto.

Restricción 2: Sólo una de las dos plantas debe asignarse para la producción de los nuevos productos.

En esencia, el costo unitario de producción de cada producto sería el mismo en las dos plantas. Sin embargo, por diferencias en las instalaciones, el número de horas de producción por unidad de cada producto puede diferir entre ellas. Estos datos se dan en la tabla 11.2, junto con otra información relevante, que incluye las estimaciones del departamento de mercadotecnia sobre el número máximo de unidades de cada producto que podrían venderse a la semana. El objetivo es seleccionar los productos, la planta y las tasas de producción de los bienes elegidos de manera que se maximice la ganancia total.

■ TABLA 11.2 Datos del ejemplo 1 (problema de la Good Products Co.)

	Tiempo de producción utilizado por cada unidad producida			Horas disponibles por semana
	Producto 1	Producto 2	Producto 3	
Planta 1	3 horas	4 horas	2 horas	30 horas
Planta 2	4 horas	6 horas	2 horas	
Ganancia unitaria	5	7	3	(miles de dólares)
Ventas potenciales	7	5	9	(unidades por semana)

¹ Se puede consultar una evidencia que soporta esta conclusión en J. H. Owen y S. Mehrotra, “On the Value of Binary Expansions for General Mixed Integer Linear Programs”, en *Operations Research*, **50**: 810-819, 2002.

De alguna manera este problema se parece al *problema de mezcla de productos* estándar del ejemplo de la Wyndor Glass Co., que se describió en la sección 3.1. En realidad, si se eliminaran las dos restricciones anteriores y se requiriera que cada unidad de producto usara las horas de producción que se proporcionan en la tabla 11.2 en *ambas plantas*, se convertiría en un problema tipo. En particular, sean x_1 , x_2 y x_3 las tasas de producción de los respectivos productos, el modelo sería

$$\text{Maximizar} \quad Z = 5x_1 + 7x_2 + 3x_3,$$

sujeta a

$$3x_1 + 4x_2 + 2x_3 \leq 30$$

$$4x_1 + 6x_2 + 2x_3 \leq 40$$

$$x_1 \leq 7$$

$$x_2 \leq 5$$

$$x_3 \leq 9$$

y

$$x_1 \geq 0, \quad x_2 \geq 0, \quad x_3 \geq 0.$$

No obstante, cuando se trata de problemas reales, la limitación 1 necesita que se agregue al modelo la siguiente restricción:

El número de variables de decisión estrictamente positivas (x_1, x_2, x_3) debe ser ≤ 2 .

Esta restricción no se ajusta al formato de programación lineal o entera, por lo que la pregunta clave es cómo convertirla al formato que permita usar un algoritmo eficiente para resolver el modelo completo. Si las variables de decisión fueran binarias, entonces la restricción se expresaría como $x_1 + x_2 + x_3 \leq 2$. Sin embargo, con variables de decisión *continuas*, se necesita un enfoque más complicado que incluya la introducción de variables binarias auxiliares.

La limitación 2 necesita que se sustituyan las dos primeras restricciones funcionales ($3x_1 + 4x_2 + 2x_3 \leq 30$ y $4x_1 + 6x_2 + 2x_3 \leq 40$) por la restricción

$$\text{ya sea} \quad 3x_1 + 4x_2 + 2x_3 \leq 30$$

$$\text{o bien} \quad 4x_1 + 6x_2 + 2x_3 \leq 40$$

debe cumplirse, donde la elección de cuál restricción se cumple corresponde a la elección de qué planta se usará para producir los nuevos productos. En la sección anterior se analizó cómo se puede convertir una restricción de una o la otra al formato de programación lineal o entera, de nuevo con la ayuda de variables binarias auxiliares.

Formulación con variables binarias auxiliares. Para manejar la limitación 1, se introducen tres variables binarias auxiliares (y_1, y_2, y_3) con la interpretación

$$y_j = \begin{cases} 1 & \text{si } x_j > 0 \text{ se puede cumplir (se puede producir } j) \\ 0 & \text{si } x_j = 0 \text{ debe cumplirse (no se puede producir } j), \end{cases}$$

para $j = 1, 2, 3$. Para incluir esta interpretación en el modelo con la ayuda de M —un número positivo muy grande—, se agregan las restricciones

$$x_1 \leq My_1$$

$$x_2 \leq My_2$$

$$x_3 \leq My_3$$

$$y_1 + y_2 + y_3 \leq 2$$

$$y_j \text{ es binaria, para } j = 1, 2, 3.$$

La restricción de una o la otra y las restricciones de no negatividad proporcionan una región *acotada* para las variables de decisión, con lo que cada $x_j \leq M$ en toda la región. Por tanto, para cada $x_j \leq My_j$ la restricción $y_j = 1$ permite que x_j tome cualquier valor en la región factible, mientras que $y_j = 0$ fuerza a que $x_j = 0$. (De manera inversa, $x_j > 0$ hace que $y_j = 1$, mientras que $x_j = 0$ permite que y_j tome cualquiera de los dos valores.) En consecuencia, cuando la cuarta restricción hace que

a lo más dos de las y_j sean iguales a 1, significa que es posible elegir, cuando mucho, fabricar dos de los nuevos productos.

Para manejar la limitación 2 se introduce otra variable binaria auxiliar y_4 con la interpretación

$$y_4 = \begin{cases} 1 & \text{si } 4x_1 + 6x_2 + 2x_3 \leq 40 \text{ debe cumplirse (se elige la planta 2)} \\ 0 & \text{si } 3x_1 + 4x_2 + 2x_3 \leq 30 \text{ debe cumplirse (se elige la planta 1).} \end{cases}$$

Como se vio en la sección 11.3, esta interpretación se toma en cuenta al agregar las restricciones,

$$\begin{aligned} 3x_1 + 4x_2 + 2x_3 &\leq 30 + My_4 \\ 4x_1 + 6x_2 + 2x_3 &\leq 40 + M(1 - y_4) \\ y_4 &\text{ es binaria.} \end{aligned}$$

En consecuencia, después de mover todas las variables al lado izquierdo de las restricciones, el modelo completo es

$$\text{Maximizar } Z = 5x_1 + 7x_2 + 3x_3,$$

sujeta a

$$\begin{aligned} x_1 &\leq 7 \\ x_2 &\leq 5 \\ x_3 &\leq 9 \\ x_1 - My_1 &\leq 0 \\ x_2 - My_2 &\leq 0 \\ x_3 - My_3 &\leq 0 \\ y_1 + y_2 + y_3 &\leq 2 \\ 3x_1 + 4x_2 + 2x_3 - My_4 &\leq 30 \\ 4x_1 + 6x_2 + 2x_3 + My_4 &\leq 40 + M \end{aligned}$$

y

$$\begin{aligned} x_1 &\geq 0, & x_2 &\geq 0, & x_3 &\geq 0 \\ y_j &\text{ es binaria,} & & & \text{ para } j = 1, 2, 3, 4. \end{aligned}$$

Éste es ahora un modelo de PEM, con tres variables (las x_j) que no tienen que ser enteras y cuatro variables binarias, por lo cual es posible usar un algoritmo de PEM para resolverlo. Luego de ello —después de sustituir M por un valor numérico grande—,² la solución óptima es $y_1 = 1$, $y_2 = 0$, $y_3 = 1$, $y_4 = 1$, $x_1 = 5\frac{1}{2}$, $x_2 = 0$ y $x_3 = 9$; es decir, se eligen los productos 1 y 3 para producción, se elige la planta 2 y las tasas de producción deben ser $5\frac{1}{2}$ unidades por semana del producto 1 y 9 unidades por semana del producto 3. La ganancia total resultante es de \$54 500 semanales.

EJEMPLO 2 Violación de la proporcionalidad

La CORPORACIÓN SUPERSUDS ha comenzado a desarrollar sus planes de comercialización de sus nuevos productos para el próximo año. Considera la posibilidad de comprar un total de cinco comerciales de televisión en las redes nacionales para promocionar tres de estos productos. El problema que se debe resolver es cómo asignar los cinco comerciales a estos productos, con un máximo de tres —y un mínimo de cero— para cada uno de ellos.

En la tabla 11.3 se muestra el efecto estimado de asignar 0, 1, 2 o 3 comerciales a cada producto. Este efecto se mide en términos de la *ganancia* (en millones de dólares) que será resultado de las *ventas adicionales* generadas por los comerciales. El objetivo es asignar cinco comerciales a los productos de manera que se maximice la ganancia total.

²En la práctica, debe tenerse cuidado de dar un valor a M que sea lo suficientemente grande como para evitar que se eliminen soluciones factibles, pero tan pequeño como sea posible para evitar una región factible innecesariamente grande para el relajamiento de PL (descrito en la siguiente sección) y para evitar inestabilidad numérica. El examen cuidadoso de las restricciones en este ejemplo revela que el valor factible mínimo de M es $M = 9$.

Este pequeño problema se puede resolver con facilidad con programación dinámica (capítulo 10) o incluso por inspección. (La solución óptima es asignar dos comerciales al producto 1, ninguno al producto 2 y 3 al producto 3). Sin embargo, como ilustración, se desarrollarán dos formulaciones diferentes de PEB. Este tipo de formulación sería necesaria si este pequeño problema tuviera que incorporarse a un modelo de PE más grande que tal vez involucrara la asignación de recursos a las actividades de comercialización de los nuevos productos de toda la corporación.

Una formulación con variables binarias auxiliares. Una formulación natural surge si se denota por x_1, x_2, x_3 el número de comerciales de TV asignados a los respectivos productos. En consecuencia, la contribución de cada x_j a la función objetivo se presenta en la columna correspondiente de la tabla 11.3. Sin embargo, cada una de estas columnas viola el supuesto de proporcionalidad descrito en la sección 3.3, debido a lo cual no se puede establecer una función objetivo *lineal* en términos de estas variables de decisión enteras.

A continuación, observe lo que ocurre cuando se introducen *variables binarias auxiliares* y_{ij} en cada valor entero positivo de $x_i = j$ ($j = 1, 2, 3$), donde y_{ij} se interpreta como

$$y_{ij} = \begin{cases} 1 & \text{si } x_i = j \\ 0 & \text{de otra manera.} \end{cases}$$

(Por ejemplo, $y_{21} = 0$, $y_{22} = 0$ y $y_{23} = 1$, significa que $x_2 = 3$.) El modelo de PEB *lineal* que se obtiene es

$$\text{Maximizar} \quad Z = y_{11} + 3y_{12} + 3y_{13} + 2y_{22} + 3y_{23} - y_{31} + 2y_{32} + 4y_{33},$$

sujeta a

$$\begin{aligned} y_{11} + y_{12} + y_{13} &\leq 1 \\ y_{21} + y_{22} + y_{23} &\leq 1 \\ y_{31} + y_{32} + y_{33} &\leq 1 \\ y_{11} + 2y_{12} + 3y_{13} + y_{21} + 2y_{22} + 3y_{23} + y_{31} + 2y_{32} + 3y_{33} &= 5 \end{aligned}$$

y

todas las y_{ij} es binaria.

Observe que las tres primeras restricciones funcionales aseguran que a cada x_i se le asignará exactamente uno de sus valores posibles. (Aquí, $y_{i1} + y_{i2} + y_{i3} = 0$ corresponde a $x_i = 0$, lo que no contribuye en nada a la función objetivo.) La última restricción funcional asegura que $x_1 + x_2 + x_3 = 5$. En consecuencia, de acuerdo con la tabla 11.3, la función objetivo *lineal* proporciona la ganancia total.

Al resolver este modelo de PEB se obtiene la siguiente solución óptima

$$\begin{array}{llll} y_{11} = 0, & y_{12} = 1, & y_{13} = 0, & \text{es decir, } x_1 = 2 \\ y_{21} = 0, & y_{22} = 0, & y_{23} = 0, & \text{es decir, } x_2 = 0 \\ y_{31} = 0, & y_{32} = 0, & y_{33} = 1, & \text{es decir, } x_3 = 3. \end{array}$$

■ **TABLA 11.3** Datos del ejemplo 2
(problema de la
Supersuds Corp.)

Número de comerciales	Ganancia		
	Producto		
	1	2	3
0	0	0	0
1	1	0	-1
2	3	2	2
3	3	3	4

Otra formulación con variables binarias auxiliares. Se vuelven a definir las variables binarias auxiliares anteriores y_{ij} de la siguiente manera:

$$y_{ij} = \begin{cases} 1 & \text{si } x_i \geq j \\ 0 & \text{de otra manera.} \end{cases}$$

Entonces, la diferencia estriba en que ahora $y_{ij} = 1$ si $x_i \geq j$ en lugar de que $x_i = j$. Entonces,

$$\begin{array}{llll} x_i = 0 & \Rightarrow & y_{i1} = 0, & y_{i2} = 0, & y_{i3} = 0, \\ x_i = 1 & \Rightarrow & y_{i1} = 1, & y_{i2} = 0, & y_{i3} = 0, \\ x_i = 2 & \Rightarrow & y_{i1} = 1, & y_{i2} = 1, & y_{i3} = 0, \\ x_i = 3 & \Rightarrow & y_{i1} = 1, & y_{i2} = 1, & y_{i3} = 1, \end{array}$$

de manera que $x_i = y_{i1} + y_{i2} + y_{i3}$

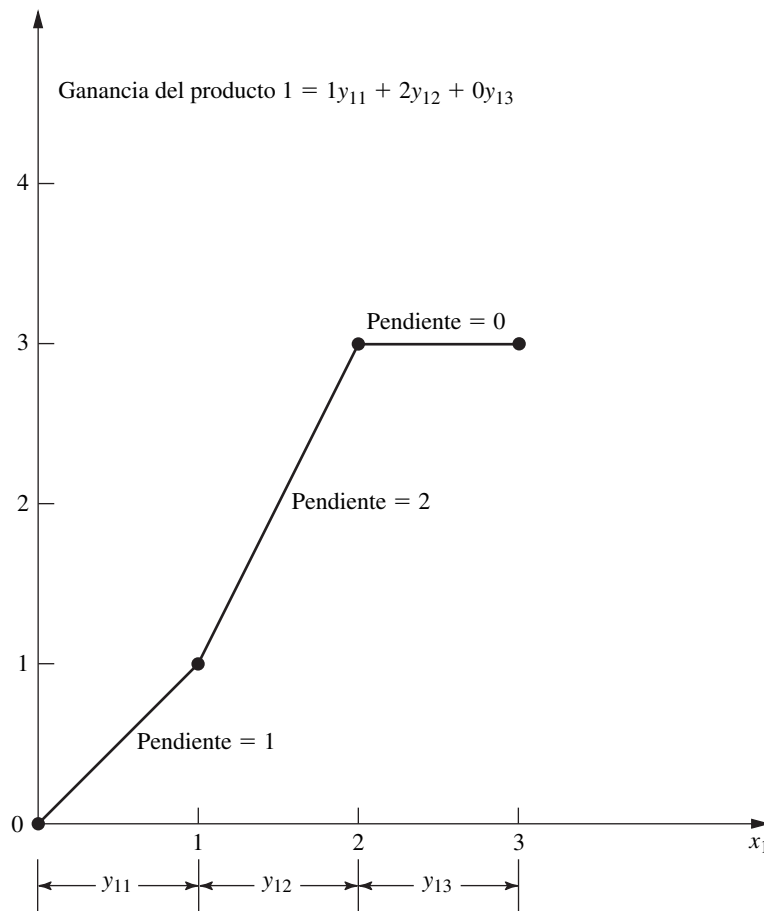
para $i = 1, 2, 3$. Se refuerzan las definiciones de que se permite que $y_{i2} = 1$ sea contingente respecto de $y_{i1} = 1$, y que $y_{i3} = 1$ lo es respecto de $y_{i2} = 1$, por medio las restricciones

$$y_{i2} \leq y_{i1} \quad \text{y} \quad y_{i3} \leq y_{i2}, \quad \text{para } i = 1, 2, 3.$$

La nueva definición de y_{ij} cambia también la función objetivo, como se ilustra en la figura 11.1, en la parte que corresponde al producto 1 de esta función. Como y_{11} , y_{12} y y_{13} proporcionan los incrementos sucesivos (si los hay) del valor de x_1 (a partir de un valor de 0), los coeficientes de y_{11} , y_{12} y y_{13} están dados por los incrementos respectivos en la columna del producto 1 de la tabla 11.3 ($1 - 0 = 1$, $3 - 1 = 2$, $3 - 3 = 0$). Estos incrementos son las *pendientes* en la figura 11.1, que conducen a $1y_{11} + 2y_{12} + 0y_{13}$ para la parte del producto 1 en la función objetivo. Observe que la aplicación de este enfoque a los tres productos conduce a una función objetivo *lineal*.

■ FIGURA 11.1

Utilidad obtenida por las ventas adicionales del producto 1 generada por x_1 anuncios de televisión, donde las pendientes dadas corresponden a los coeficientes de la función objetivo de la segunda formulación de PEB del ejemplo 2 (problema de la Supersuds Corp.).



Después de mover todas las variables al lado izquierdo de las restricciones el modelo completo de PEB que se obtiene es el siguiente:

$$\text{Maximizar } Z = y_{11} + 2y_{12} + 2y_{22} + y_{23} - y_{31} + 3y_{32} + 2y_{33},$$

sujeta a

$$y_{12} - y_{11} \leq 0$$

$$y_{13} - y_{12} \leq 0$$

$$y_{22} - y_{21} \leq 0$$

$$y_{23} - y_{22} \leq 0$$

$$y_{32} - y_{31} \leq 0$$

$$y_{33} - y_{32} \leq 0$$

$$y_{11} + y_{12} + y_{13} + y_{21} + y_{22} + y_{23} + y_{31} + y_{32} + y_{33} = 5$$

y

todas las y_{ij} son binarias.

Al resolver este modelo de PEB se obtiene la siguiente solución óptima

$$\begin{array}{llll} y_{11} = 1, & y_{12} = 1, & y_{13} = 0, & \text{es decir, } x_1 = 2 \\ y_{21} = 0, & y_{22} = 0, & y_{23} = 0, & \text{es decir, } x_2 = 0 \\ y_{31} = 1, & y_{32} = 1, & y_{33} = 1, & \text{es decir, } x_3 = 3. \end{array}$$

Existen pocos argumentos para elegir entre este modelo de PEB y el anterior excepto el gusto personal. Tienen el mismo número de variables binarias, una consideración primordial cuando se determina el esfuerzo computacional en este tipo de problemas. Ambos tienen alguna *estructura especial*—restricciones para *alternativas mutuamente excluyentes* en el primer modelo y restricciones para las *decisiones contingentes* en el segundo—que puede ayudar a resolverlo con mayor rapidez. El segundo modelo tiene más restricciones funcionales que el primero.

EJEMPLO 3 Cobertura de todas las características

SOUTHWESTERN AIRWAYS necesita asignar sus tripulaciones para cubrir todos sus vuelos programados. Se estudiará el problema de asignar tres tripulaciones con base en San Francisco (SF) a los vuelos enumerados en la tabla 11.4. Las otras 12 columnas muestran 12 secuencias de vuelos factibles de una tripulación. (Los números en cada columna indican el orden de los vuelos.) Es necesario elegir tres de estas secuencias (una por tripulación) de tal manera que se cubran todos los vuelos. (Se permite tener más de una tripulación en un vuelo, en el cual los miembros de la tripulación adicional volarían como pasajeros, pero los contratos colectivos de trabajo requieren que se pague el tiempo de la tripulación adicional como si estuviera en horario de trabajo.) El costo de asignar una tripulación a una secuencia de vuelos específica se muestra (en miles de dólares) en el renglón inferior de la tabla. El objetivo es minimizar el costo total de asignar las tres tripulaciones de manera que cubran todos los vuelos.

Formulación con variables binarias. Con 12 secuencias factibles de vuelos, se deben enfrentar 12 decisiones sí o no:

¿Debe asignarse la secuencia j a la tripulación? ($j = 1, 2, \dots, 12$)

Por tanto, se usan 12 variables binarias para representar las respectivas decisiones:

$$x_j = \begin{cases} 1 & \text{si se asigna la secuencia } j \text{ a una tripulación} \\ 0 & \text{de otra manera.} \end{cases}$$

La parte más interesante de esta formulación es la naturaleza de cada restricción que asegura que se cubra el vuelo correspondiente. Por ejemplo, considere el último vuelo de la tabla 11.4 [Seattle a Los Ángeles (LA)]. Cinco secuencias —es decir, las secuencias 6, 9, 10, 11 y 12— in-

■ **TABLA 11.4** Datos del ejemplo 3 (el problema de la aerolínea Southwestern)

Vuelo	Secuencia factible de vuelos											
	1	2	3	4	5	6	7	8	9	10	11	12
1. San Francisco a Los Ángeles	1			1			1			1		
2. San Francisco a Denver		1			1			1			1	
3. San Francisco a Seattle			1			1			1			1
4. Los Ángeles a Chicago				2			2		3	2		3
5. Los Ángeles a San Francisco	2					3				5	5	
6. Chicago a Denver				3	3				4			
7. Chicago a Seattle							3	3		3	3	4
8. Denver a San Francisco		2		4	4				5			
9. Denver a Chicago					2			2			2	
10. Seattle a San Francisco			2				4	4				5
11. Seattle a Los Ángeles						2			2	4	4	2
Costo, en miles de dólares	2	3	4	6	7	5	7	8	9	9	8	9

cluyen este vuelo. Por tanto, debe elegirse al menos una de estas secuencias. La restricción que resulta es

$$x_6 + x_9 + x_{10} + x_{11} + x_{12} \geq 1.$$

Con restricciones similares para los otros 10 vuelos, el modelo completo de PEB es

$$\begin{aligned} \text{Minimizar} \quad Z = & 2x_1 + 3x_2 + 4x_3 + 6x_4 + 7x_5 + 5x_6 + 7x_7 + 8x_8 + 9x_9 \\ & + 9x_{10} + 8x_{11} + 9x_{12}, \end{aligned}$$

sujeta a

$$\begin{aligned} x_1 + x_4 + x_7 + x_{10} &\geq 1 && (\text{SF a LA}) \\ x_2 + x_5 + x_8 + x_{11} &\geq 1 && (\text{SF a Denver}) \\ x_3 + x_6 + x_9 + x_{12} &\geq 1 && (\text{SF a Seattle}) \\ x_4 + x_7 + x_9 + x_{10} + x_{12} &\geq 1 && (\text{LA a Chicago}) \\ x_1 + x_6 + x_{10} + x_{11} &\geq 1 && (\text{LA a SF}) \\ x_4 + x_5 + x_9 &\geq 1 && (\text{Chicago a Denver}) \\ x_7 + x_8 + x_{10} + x_{11} + x_{12} &\geq 1 && (\text{Chicago a Seattle}) \\ x_2 + x_4 + x_5 + x_9 &\geq 1 && (\text{Denver a SF}) \\ x_5 + x_8 + x_{11} &\geq 1 && (\text{Denver a Chicago}) \\ x_3 + x_7 + x_8 + x_{12} &\geq 1 && (\text{Seattle a SF}) \\ x_6 + x_9 + x_{10} + x_{11} + x_{12} &\geq 1 && (\text{Seattle a LA}) \\ \sum_{j=1}^{12} x_j &= 3 && (\text{asignar tres tripulaciones}) \end{aligned}$$

y

$$x_j \text{ es binaria, para } j = 1, 2, \dots, 12.$$

Una solución óptima para este modelo de PEB es

$$\begin{aligned} x_3 &= 1 && (\text{asignar la secuencia 3 a una tripulación}) \\ x_4 &= 1 && (\text{asignar la secuencia 4 a una tripulación}) \\ x_{11} &= 1 && (\text{asignar la secuencia 11 a una tripulación}) \end{aligned}$$

y todas las demás $x_j = 0$, con un costo total de \$18 000 (otra solución óptima es $x_1 = 1$, $x_5 = 1$, $x_{12} = 1$, y todas las demás $x_j = 0$).

Este ejemplo ilustra una clase más amplia de problemas llamada **problemas de cobertura de conjunto**.³ Cualquier problema de cobertura se puede describir en términos generales como uno que involucra a cierto número de *actividades* potenciales (como las secuencias de vuelo) y *características* (como los vuelos). Cada actividad posee alguna pero no todas las características. El objetivo es determinar la combinación menos costosa de actividades que de manera colectiva posea (cubra) todas las características, por lo menos una vez. Así, si S_i es el conjunto de todas las actividades que poseen la característica i , al menos un elemento de S_i debe quedar incluido entre las actividades seleccionadas, por lo que la restricción,

$$\sum_{j \in S_i} x_j \geq 1,$$

se incluye en cada característica i .

Una clase de problemas relacionada, conocida como **problemas de partición de conjuntos**, cambia todas las restricciones de este tipo a

$$\sum_{j \in S_i} x_j = 1,$$

por lo que debe incluirse *exactamente* un elemento de cada conjunto S_i entre las actividades seleccionadas. En el ejemplo de la programación de tripulaciones, esto significa que cada vuelo debe estar incluido exactamente una vez entre las secuencias elegidas, lo que elimina la posibilidad de llevar tripulaciones adicionales (como pasajeros) en cualquier vuelo.

11.5 ALGUNAS PERSPECTIVAS ACERCA DE LA SOLUCIÓN DE PROBLEMAS DE PROGRAMACIÓN ENTERA

Puede parecer que la solución de problemas de PE es sencilla. Después de todo, los problemas de *programación lineal* se pueden resolver en forma eficiente, y la única diferencia es que la PE tiene muchas menos soluciones que considerar. En realidad, se garantiza que los problemas de PE *pura* con región factible acotada tengan sólo un número *finito* de soluciones factibles.

Desafortunadamente, existen dos falacias en esta línea de razonamiento. Una es que tener un número finito de soluciones factibles asegura que el problema se puede resolver. Los números finitos pueden ser astronómicamente grandes. Por ejemplo, considere el caso sencillo de problemas de PEB. Si se tienen n variables, existen 2^n soluciones que considerar (algunas de las cuales se pueden descartar por violar las restricciones funcionales). En consecuencia, cada vez que n aumenta en 1, el número de soluciones se *duplica*. Este patrón se llama **crecimiento exponencial** de la dificultad del problema. Con $n = 10$, existen más de mil soluciones (1 024); con $n = 20$, son más de un millón; con $n = 30$ resultan más de mil millones, y así sucesivamente; en razón de ello, aun las computadoras más eficientes son incapaces de realizar una enumeración exhaustiva —que verifique la factibilidad de cada solución y, de ser factible, que calcule el valor de la función objetivo— en problemas de PEB con unas cuantas docenas de variables, sin mencionar los problemas de PE *general* con el mismo número de variables enteras. Por fortuna, empezando con las ideas que se describirán en secciones subsecuentes, en la actualidad los mejores algoritmos de PE son ampliamente superiores al método de la numeración exhaustiva. La mejora en las dos décadas pasadas ha sido sorprendente. Los problemas PEB que requerían años de procesamiento para resolverse hace 20 años, en la actualidad pueden resolverse en segundos mediante el uso de paquetes de software comerciales (como, por ejemplo, el paquete CPLEX). Este aumento dramático en la velocidad se debe al enorme progreso en tres áreas: mejoras significativas en los algoritmos PEB (así como en otros algoritmos de PE), mejoras sorprendentes en algoritmos de programación lineal muy utilizados en los algoritmos de programación entera y el notable incremento de la velocidad de las computadoras (entre las cuales se incluyen las de escritorio). Como consecuencia de todo ello,

³De manera estricta, un problema de cobertura de conjunto no incluye *otras* restricciones funcionales como la última del ejemplo de programación de tripulaciones. Algunas veces se supone que todos los coeficientes de la función objetivo que se quiere minimizar son iguales a *uno*, y entonces cuando este supuesto no se cumple se usa el nombre de *problema de cobertura de conjunto ponderada*.

Recuadro de aplicación

Taco Bell Corporation cuenta con más de 6 500 restaurantes de servicio rápido en Estados Unidos y un mercado internacional que cada día crece más. Sirve alrededor de 2 000 millones de alimentos anualmente, lo cual genera aproximadamente 5.4 miles de millones de dólares de ingreso de ventas al año.

En cada restaurante Taco Bell, la cantidad de servicios es muy variable a lo largo del día (y de un día a otro), con una alta concentración durante las horas de comida. Por tanto, la determinación de cuántos empleados deben programarse para llevar a cabo qué funciones en el restaurante en todo momento, representa un problema complejo y tedioso.

Para enfrentar este problema, la gerencia de Taco Bell giró órdenes a un equipo de investigación de operaciones (que incluía a varios consultores) para que desarrollaran un nuevo sistema de administración del trabajo. El equipo concluyó que el sistema necesitaba tres componentes principales: 1) un *modelo de pronóstico* para predecir las transacciones de los clientes en cualquier momento, 2) un *modelo de simulación* (como los que se describen en el capítulo 20) para traducir las transacciones de los clientes en necesidades de trabajo y, 3) un *modelo de programación entera* con el fin de programar a los empleados para satisfacer las necesidades de trabajo y minimizar la nómina.

Las variables de decisión enteras de este modelo de programación entera para cualquier restaurante son el número de empleados asignados en cada turno que inicien su horario de trabajo en determinados tiempos específicos. La duración de estos turnos son también variables de decisión (restrin-

gidas a estar entre la duración máxima y mínima permisible de un turno), pero en este caso variables de decisión *continuas*, por lo que el modelo es un modelo IP mezclado. Las restricciones principales especifican que el número de empleados que trabajan durante cada intervalo de 15 minutos debe ser mayor o igual al número mínimo que se requiere durante ese intervalo (de acuerdo con el modelo de pronóstico).

Este modelo PEM es parecido al *modelo de programación lineal* para asignar empleados a los turnos en el problema de United Airlines que se describió en el recuadro de aplicación de la sección 3.4. Sin embargo, la diferencia fundamental estriba en que el número de empleados que trabaja en turnos en los restaurantes Taco Bell es mucho menor que el de United Airlines, por lo que es necesario restringir dichas variables de decisión a valores enteros en el caso del modelo de Taco Bell (mientras que los valores no enteros de una solución en el caso de United Airlines puede redondearse con facilidad a valores enteros con tan sólo una pequeña pérdida de precisión).

La implantación de este modelo MIP junto con los demás componentes del sistema de administración del trabajo ha brindado a Taco Bell *ahorros documentados de 13 millones de dólares al año* en costos laborales.

Fuente: J. Hueter y W. Swart: "An Integrated Labor-Management System for Taco Bell", en *Interfaces*, 28(1): 75-91, enero-febrero de 1998. (En el sitio en internet de este libro, www.mhhe.com/hillier, se proporciona una liga hacia este artículo.)

una gran cantidad de problemas PEB de gran tamaño están siendo resueltos en la actualidad, lo que hubiera sido imposible en las décadas pasadas. Los mejores algoritmos de hoy en día son capaces de resolver *algunos* problemas PEB puros con cientos de miles de variables. Sin embargo, debido al *crecimiento exponencial*, ni los mejores algoritmos garantizan la solución de todos los problemas relativamente pequeños (con menos de 100 variables binarias o enteras). Según sus características, ciertos problemas pequeños pueden ser más difíciles de resolver que algunos más grandes.

Cuando se trata con variables enteras generales en lugar de variables binarias, el tamaño de los problemas que pueden resolverse tiende a ser mucho más pequeño. Sin embargo, hay excepciones. Por ejemplo, hace varios años, la versión profesional de CPLEX 8.0 resolvió de manera exitosa un problema de PE con 215 mil variables enteras generales, 75 mil restricciones funcionales y 6 millones de coeficientes de restricción distintos de cero. Con todo, las versiones más actuales de CPLEX son mucho más poderosas.

La segunda falacia es que si se eliminan algunas soluciones factibles (las no enteras) de un problema de programación lineal, será más fácil resolverlo. Por el contrario, sólo cuando todas estas soluciones factibles están ahí, se puede garantizar (vea la sección 5.1) que existe una solución factible en el vértice (FEV) [y por ende la solución básica factible (BF) correspondiente] que es óptima para el problema completo. Esta garantía es la clave de la extraordinaria eficiencia del método *símplex*. Como resultado, en general es mucho más sencillo resolver los problemas de programación lineal que los de programación entera.

Es lógico, entonces, que la mayor parte de los algoritmos exitosos de programación entera incorporen un algoritmo de programación lineal, como el método *símplex* (o el método *símplex dual*) lo más que puedan, y que relacionen partes del problema de PE bajo consideración con el problema correspondiente de programación lineal (es decir, el mismo problema con la restricción de valores enteros eliminada). En cualquier problema dado de PE, el problema correspondiente de programación lineal se conoce como su **relajamiento de PL**. El algoritmo que se presenta en las dos secciones siguientes ilustra cómo se puede usar una sucesión de relajamientos de PL para

porciones de un problema de programación entera, con objeto de resolver de manera eficaz un problema completo de PE.

Existe una situación especial en la que no es más difícil resolver el problema de PE que resolver una vez su relajamiento de PL al aplicar el método símplex; en este caso, la solución del relajamiento de PL satisface la restricción de valores enteros del problema de PE. Cuando ocurre esto, la solución también *debe* ser óptima para el problema de PE, puesto que se trata de la mejor solución entre todas las soluciones factibles del relajamiento de PL, que incluyen todas las soluciones factibles del problema de PE. Entonces, es normal que un algoritmo de programación entera comience con la aplicación del método símplex a su relajamiento de PL, para verificar si tiene lugar este acontecimiento fortuito.

Aunque, sin duda, casi siempre es accidental que la solución óptima del relajamiento de PL sea entera, en realidad existen varios *tipos especiales* de problemas de PE para los que este resultado se puede *garantizar*. Ya se vieron dos de estos casos especiales en los capítulos 8 y 9; éstos son el *problema del flujo de costo mínimo* (con parámetros enteros) y sus casos especiales (que incluyen el *problema de transporte*, el *problema de trasbordo*, el *problema de asignación*, el *problema de la ruta más corta* y el *de flujo máximo*). Esta garantía se puede dar en razón de la *estructura especial* que poseen estos tipos particulares de problemas —por ejemplo, vea la tabla 8.6—, que asegura que toda solución factible es entera, como se estableció en la propiedad de soluciones enteras que se presentó en las secciones 8.1 y 9.6. Por ello, estos tipos especiales de problemas de PE se pueden manejar como problemas de programación lineal, puesto que se pueden resolver por completo al aplicar las versiones simplificadas del método símplex.

Aunque toda esta simplificación no es usual, con frecuencia los problemas de PE que surgen en la práctica tienen *alguna* estructura especial que se puede aprovechar para simplificarlos. (Los ejemplos 2 y 3 de la sección anterior caen en esta categoría debido a sus restricciones de *alternativas mutuamente excluyentes* o las de *decisiones contingentes* o las restricciones de *cobertura de un conjunto*.) A veces se pueden resolver con éxito versiones muy grandes de estos problemas. Cada vez son más importantes los algoritmos especiales diseñados específicamente para explotar ciertos tipos de estructuras especiales de programación entera.

En consecuencia, los tres factores determinantes de la *dificultad computacional* de un problema de PE son 1) el *número de variables enteras*, 2) si estas variables enteras son *binarias* o *generales* y 3) cualquier *estructura especial* del problema. Esta situación es opuesta a la de programación lineal, en donde el número de restricciones (funcionales) es mucho más importante que el número de variables. En programación entera, el número de restricciones tiene *alguna* importancia —en especial si se van a resolver los relajamientos de PL—, pero es estrictamente secundario en el caso de los otros tres factores. En realidad, existen casos en los que *aumentar* el número de restricciones *disminuye* el tiempo de cálculo, puesto que se reduce el número de soluciones factibles. En los problemas de PEM es el número de variables *enteras* y no el número *total* de variables lo que es importante, pues las variables *continuas* casi no tienen efecto sobre el esfuerzo computacional.

Debido a que, en general, es mucho más difícil resolver los problemas de PE que los de programación lineal, a veces es tentador usar el procedimiento aproximado y aplicar el método símplex al relajamiento de PL y después *redondear* los valores no enteros a enteros en la solución que se obtuvo. Este enfoque puede ser adecuado en algunas aplicaciones, en especial si los valores de las variables son tan grandes que el redondeo introduce un error muy pequeño; sin embargo, debe tenerse cuidado al poner en práctica este procedimiento, pues se corren dos riesgos.

Una desventaja es que una solución óptima de programación lineal *no necesariamente es factible* después de redondearla. Con frecuencia es difícil decidir en qué sentido redondear para conservar la factibilidad. Incluso, podría ser necesario cambiar el valor de algunas variables en una o más unidades después de redondear. Para ilustrar este método, considere el siguiente problema:

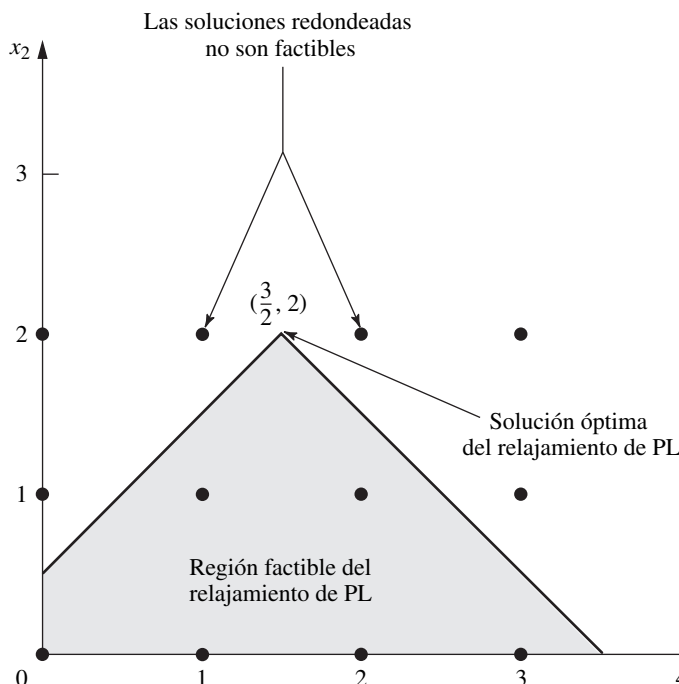
$$\text{Maximizar} \quad Z = x_2,$$

sujeta a

$$\begin{aligned} -x_1 + x_2 &\leq \frac{1}{2} \\ x_1 + x_2 &\leq 3\frac{1}{2} \end{aligned}$$

■ FIGURA 11.2

Ejemplo de un problema de PE donde la solución óptima del relajamiento de PL no se puede redondear de ninguna manera que conserve la factibilidad.



y

$$x_1 \geq 0, \quad x_2 \geq 0$$

$$x_1, x_2 \text{ son enteros.}$$

Como se muestra en la figura 11.2, la solución óptima para el relajamiento de PL es $x_1 = 1\frac{1}{2}, x_2 = 2$, pero es imposible redondear la variable no entera x_1 a 1 o 2 (o a cualquier otro entero) y conservar la factibilidad. Esta factibilidad sólo se puede conservar si también se cambia el valor entero de x_2 . Es fácil imaginar la complejidad a la que puede llevar este tipo de dificultades cuando se tiene en frente decenas o cientos de restricciones y variables.

Aun cuando una solución óptima para el relajamiento de PL se pueda redondear con éxito, todavía queda otra dificultad. No existe garantía de que esta solución redondeada sea la solución óptima de programación entera. En realidad, puede ser que se encuentre muy lejos del óptimo en términos del valor de la función objetivo. El siguiente problema ejemplifica este hecho:

$$\text{Maximizar} \quad Z = x_1 + 5x_2,$$

sujeta a

$$x_1 + 10x_2 \leq 20$$

$$x_1 \leq 2$$

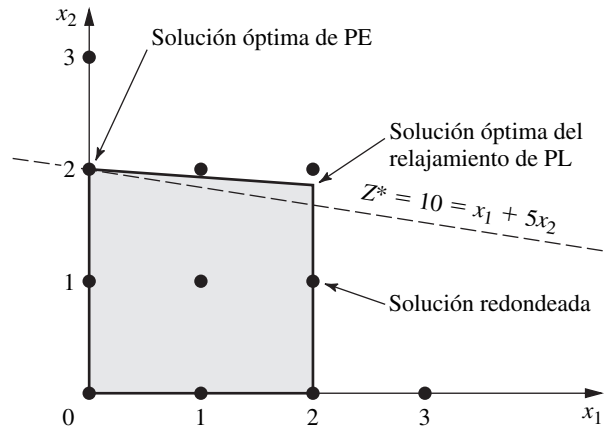
y

$$x_1 \geq 0, \quad x_2 \geq 0$$

$$x_1, x_2 \text{ son enteros.}$$

Como sólo se tienen dos variables de decisión, este problema se puede graficar como se muestra en la figura 11.3. Se puede usar esta gráfica o el método símplex para encontrar que la solución óptima del relajamiento de PL es $x_1 = 2, x_2 = \frac{9}{5}$, con $Z = 11$. Si no se dispusiera de una solución gráfica óptima —que sería el caso con más variables de decisión—, la variable con el valor no entero $x_2 = \frac{9}{5}$ se redondearía en la dirección factible a $x_2 = 1$. La solución entera que se obtendría sería $x_1 = 2, x_2 = 1$, de donde se obtiene $Z = 7$. Observe que esta solución está muy lejos de la óptima $(x_1, x_2) = (0, 2)$ donde $Z = 10$.

■ **FIGURA 11.3**
Ejemplo donde la solución del relajamiento de PL redondeada está muy lejos de ser óptima para el problema de PE.



Debido a estos dos riesgos, una mejor forma de tratar con problemas de programación entera grandes es usar uno de los *algoritmos heurísticos* disponibles. Estos algoritmos son muy eficientes para solucionar problemas grandes, pero no garantizan que se llegue a una solución óptima. Sin embargo, tienden a ser mucho más eficaces para encontrar excelentes soluciones factibles que el enfoque de redondeo que se acaba de analizar.

Uno de los desarrollos en particular emocionante de IO en años recientes ha sido el avance rápido en la elaboración de algoritmos heurísticos eficaces (llamados *metaheurísticos*) para diferentes problemas combinatorios como los de programación entera. Tres tipos sobresalientes de metaheurísticos —búsqueda de tabú, simulación de templado y algoritmos genéticos— se describirán en el capítulo 13. Estos complejos algoritmos metaheurísticos pueden incluso aplicarse a problemas de programación entera *no lineal* que tienen soluciones óptimas locales que es posible que estén lejos de la solución óptima global. También se pueden aplicar a diferentes problemas de *optimización combinatoria*, los cuales con frecuencia se pueden representar por un modelo que tiene variables enteras pero que también posee algunas restricciones que son más complicadas que las de un modelo de PE. (Tales aplicaciones se describirán más adelante en el capítulo 13.)

De regreso a la programación entera *lineal*, se dispone de un gran número de algoritmos para pequeños problemas de PE que los resuelven hasta llegar al óptimo. Sin embargo, ninguno posee una eficiencia computacional que se pueda siquiera comparar con el *método simplex* (excepto para los tipos especiales). Por tanto, el desarrollo de algoritmos de PE aún es un tema activo de investigación. Por fortuna, durante las dos últimas décadas se lograron algunos adelantos y se esperan más en los próximos años. Estos adelantos se verán en las secciones 11.8 y 11.9.

El algoritmo más popular de programación entera es la *técnica de ramificación y acotamiento* y las ideas relacionadas con la *enumeración implícita* de las soluciones factibles enteras, cuyos enfoques se analizarán más adelante. La siguiente sección presenta la técnica de ramificación y acotamiento en un contexto general y la ilustra con un algoritmo básico para enfrentar problemas de PEB. En la sección 11.7 se describe otro algoritmo del mismo tipo para problemas generales de PEM.

■ 11.6 TÉCNICA DE RAMIFICACIÓN Y ACOTAMIENTO Y SUS APLICACIONES A LA PROGRAMACIÓN ENTERA BINARIA

Dado que cualquier problema acotado de programación entera *pura* tiene sólo un número finito de soluciones factibles, resulta natural considerar el uso de algún tipo de *procedimiento de enumeración* para encontrar una solución óptima. Desafortunadamente, como se mencionó en la sección anterior, este número finito puede ser, y casi siempre es, muy grande, por lo que es imperativo que cualquier procedimiento de enumeración se estructure con habilidad para que sólo sea necesario examinar una pequeña fracción de estas soluciones factibles. Por ejemplo, la programación dinámica (vea el capítulo 10) proporciona un procedimiento de este tipo para muchos problemas que tienen un número finito de soluciones factibles (aunque no es especialmente eficiente para la mayoría de los problemas de PE). Otro enfoque de este tipo lo proporciona la *técnica de ramifi-*

cación y acotamiento. Esta técnica y algunas de sus variantes se han aplicado con cierto éxito a diversos problemas de investigación de operaciones, pero es más conocida por sus aplicaciones a los problemas de programación entera.

La idea básica en la que se apoya la técnica de ramificación y acotamiento es *divide y conquistarás*. Como es demasiado complicado resolver en forma directa el problema original “grande”, se divide en subproblemas cada vez más pequeños hasta que éstos se puedan vencer. La división (*ramificación*) se hace mediante una partición del conjunto completo de soluciones factibles en subconjuntos más pequeños. En parte, la conquista (*sondeo*) se hace mediante el *acotamiento* de la mejor solución del subconjunto para después descartar los subconjuntos cuya cota indique que no es posible que contenga una solución óptima para el problema original.

A continuación se describirá cada uno de estos pasos básicos —ramificación, acotamiento y sondeo— y se ilustrarán a través de la aplicación de un algoritmo de ramificación y acotamiento al ejemplo prototípico —el problema de California Manufacturing Co.— que se presentó en la sección 11.1 y que se repite en ésta (con las restricciones numeradas para referencias posteriores).

$$\text{Maximizar} \quad Z = 9x_1 + 5x_2 + 6x_3 + 4x_4,$$

sujeta a

$$(1) \quad 6x_1 + 3x_2 + 5x_3 + 2x_4 \leq 10$$

$$(2) \quad \quad \quad x_3 + x_4 \leq 1$$

$$(3) \quad -x_1 \quad \quad + x_3 \quad \leq 0$$

$$(4) \quad \quad -x_2 \quad \quad + x_4 \leq 0$$

y

$$(5) \quad x_j \text{ es binaria,} \quad \text{para } j = 1, 2, 3, 4.$$

Ramificación

Cuando se manejan variables binarias, la forma más sencilla de partir el conjunto de soluciones factibles es fijar el valor de una variable (por ejemplo, x_1) en $x_1 = 0$ para un subconjunto y en $x_1 = 1$ para el otro. Al hacer esto en el ejemplo prototipo, el problema completo queda dividido en dos subproblemas más pequeños, como se presentan a continuación.

Subproblema 1:

Se fija $x_1 = 0$, con lo que el subproblema que resulta es

$$\text{Maximizar} \quad Z = 5x_2 + 6x_3 + 4x_4,$$

sujeta a

$$(1) \quad 3x_2 + 5x_3 + 2x_4 \leq 10$$

$$(2) \quad \quad \quad x_3 + x_4 \leq 1$$

$$(3) \quad \quad \quad x_3 \leq 0$$

$$(4) \quad -x_2 \quad \quad + x_4 \leq 0$$

$$(5) \quad x_j \text{ es binaria,} \quad \text{para } j = 2, 3, 4.$$

Subproblema 2:

Se fija $x_1 = 1$, con lo que el subproblema que se obtiene es

$$\text{Maximizar} \quad Z = 9 + 5x_2 + 6x_3 + 4x_4,$$

sujeta a

$$(1) \quad 3x_2 + 5x_3 + 2x_4 \leq 4$$

$$(2) \quad \quad \quad x_3 + x_4 \leq 1$$

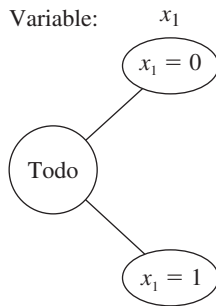
$$(3) \quad \quad \quad x_3 \leq 1$$

$$(4) \quad -x_2 \quad \quad + x_4 \leq 0$$

$$(5) \quad x_j \text{ es binaria,} \quad \text{para } j = 2, 3, 4.$$

■ FIGURA 11.4

Árbol de solución creado por la ramificación de las dos primeras iteraciones del algoritmo de ramificación y acotamiento de PEB para el ejemplo de la sección 11.1.



En la figura 11.4 se muestra esto con una división (ramificación) en subproblemas, mediante un *árbol* (definido en la sección 9.2) con *ramas* (arcos) del nodo *Todo* —correspondiente al problema completo que contiene *todas* las soluciones factibles— a los dos nodos de dos subproblemas. Este árbol, cuyas ramas “crecerán” en cada iteración, se conoce como **árbol de soluciones** (o *árbol de enumeración*) del algoritmo. La variable que se utiliza en la ramificación de una iteración cuando se asignan valores a la variable (como x_1) se llama **variable de ramificación**. (Una parte importante de algunos algoritmos de ramificación y acotamiento son ciertos métodos elaborados para seleccionar la variable de ramificación pero, por simplicidad, en esta sección se seleccionan en su orden natural: $x_1, x_2, \dots, x_n$.)

Más adelante se verá que uno de estos subproblemas se puede vencer (sondear) de inmediato, mientras que el otro requiere una nueva división en subproblemas más pequeños si se establece $x_2 = 0$ o $x_2 = 1$.

En otros problemas de PE, donde las variables enteras pueden tener más de dos valores posibles, la ramificación se puede hacer a través de establecer la variable de ramificación igual a cada valor, lo que crea más de dos subproblemas. Otro buen enfoque es especificar el *intervalo* de valores (por ejemplo, $x_j \leq 2$ o $x_j \geq 3$) para la variable de ramificación en cada nuevo subproblema. Éste es el enfoque que se usa en el algoritmo que se presenta en la sección 11.7.

Acotamiento

Ahora es necesario obtener, para cada subproblema, una *cota* que muestre el nivel de precisión de su mejor solución factible. La forma más común de hacerlo es resolver con rapidez un *relajamiento* sencillo del subproblema. Casi siempre, el **relajamiento** de un problema se obtiene *eliminando* (“relajando”) un conjunto de restricciones que dificultan obtener una solución. En los problemas de PE, las restricciones más incómodas son las que requieren que las variables sean enteras. En consecuencia, el relajamiento que más se usa es el **relajamiento de PL** que elimina este conjunto de restricciones.

Para ilustrar con un ejemplo, considere primero el problema completo que se presentó en la sección 11.1. Su relajamiento de PL se obtiene mediante la sustitución del último renglón del modelo (x_j es binaria, para $j = 1, 2, 3, 4$) por la siguiente nueva (relajada) versión de su restricción (5)

$$(5) \quad 0 \leq x_j \leq 1 \quad \text{para } j = 1, 2, 3, 4.$$

Usando el método *símplex* para resolver con rapidez su relajación PL que se obtiene de su solución óptima

$$(x_1, x_2, x_3, x_4) = \left(\frac{5}{6}, 1, 0, 1\right), \quad \text{con } Z = 16\frac{1}{2}.$$

Por tanto, $Z \leq 16\frac{1}{2}$ para todas las soluciones factibles del problema original de PEB (puesto que estas soluciones son un subconjunto de las soluciones factibles del relajamiento de PL). En realidad, como se indica en el resumen del algoritmo, esta *cota* de $16\frac{1}{2}$ se puede redondear a 16, porque todos los coeficientes de la función objetivo son enteros y, por ende, deben dar un valor entero de Z .

$$\text{Cota de todo el problema: } Z \leq 16.$$

Ahora se obtendrá, de la misma manera, las cotas de los subproblemas. En el subproblema 1, donde $x_1 = 0$, puede expresarse de manera apropiada en su relajación PL mediante la suma de la restricción que $x_1 \leq 0$ puesto que cuando se combinan con la restricción actual $0 \leq x_1 \leq 1$ obliga a que $x_1 = 0$. De manera similar, si se fija $x_1 = 1$ en el subproblema 2 nos lleva a sumar la restricción $x_1 \geq 1$ para su relajamiento de PL. Aplicando el método *símplex* obtenemos las soluciones óptimas que se muestran en seguida de estos relajamientos de PL.

$$\text{Relajamiento de PL del subproblema 1: (5) } x_1 \leq 0 \quad \text{y} \quad 0 \leq x_j \leq 1 \quad \text{para } j = 1, 2, 3, 4.$$

$$\text{Solución óptima: } (x_1, x_2, x_3, x_4) = (0, 1, 0, 1) \quad \text{con } Z = 9.$$

$$\text{Relajamiento de PL del subproblema 2: (5) } x_1 \geq 1 \quad \text{y} \quad 0 \leq x_j \leq 1 \quad \text{para } j = 1, 2, 3, 4.$$

$$\text{Solución óptima: } (x_1, x_2, x_3, x_4) = \left(1, \frac{4}{5}, 0, \frac{4}{5}\right) \quad \text{con } Z = 16\frac{1}{5}.$$

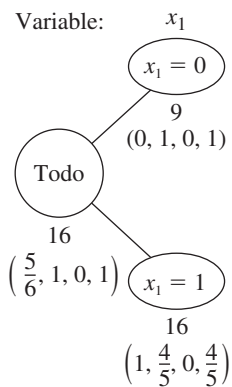
Las cotas que se obtienen son, entonces

$$\begin{aligned}\text{Cota del subproblema 1: } & Z \leq 9, \\ \text{Cota del subproblema 2: } & Z \leq 16.\end{aligned}$$

En la figura 11.5 se resumen estos resultados. En ella, los números que se encuentran exactamente abajo de los nodos son las cotas, y debajo de cada cota se presenta la solución óptima que se obtuvo para el relajamiento de PL.

■ FIGURA 11.5

Árbol de solución después de la primera iteración del algoritmo de ramificación y acotamiento de PEB del ejemplo de la sección 11.1.



Sondeo

Un subproblema se puede conquistar (sondear), y, por tanto, ya no tomarse en cuenta, en las tres formas que se describen a continuación.

Una forma se ilustra con los resultados del subproblema 1 que se dieron en el nodo $x_1 = 0$, en la figura 11.5. Observe que la solución óptima (única) de este relajamiento de PL, $(x_1, x_2, x_3, x_4) = (0, 1, 0, 1)$, es una solución *entera*. En consecuencia, esta solución debe ser también la solución óptima del subproblema 1, que debe guardarse como la primera solución **incumbente** o de apoyo (la primera solución factible que se ha encontrado hasta ahora) del problema completo, junto con su valor de Z . Este valor se denota por

$$Z^* = \text{valor de } Z \text{ de la solución de apoyo actual,}$$

de manera que en este punto $Z^* = 9$. Como esta solución se guarda, no hay razón para seguir considerando el subproblema 1 y ramificar el nodo $x_1 = 0$. Hacerlo sólo conduciría a otras soluciones factibles inferiores a la de apoyo que no son de interés. Ahora, como ya se resolvió, el subproblema 1 se **sondea** (elimina).

Los resultados anteriores sugieren una segunda prueba de sondeo importante. Como $Z^* = 9$, no existe razón alguna para tomar en cuenta ningún subproblema cuya *cota* sea ≤ 9 , puesto que tales subproblemas no pueden tener soluciones factibles mejores que la *incumbente*. Establecido en forma general, un subproblema se sondea siempre que su

$$\text{Cota} \leq Z^*.$$

Este resultado no ocurre en la iteración actual del ejemplo porque el subproblema 2 tiene una cota de 16 que es mayor que 9. No obstante, puede ocurrir más adelante, para los **descendientes** de este subproblema (nuevos problemas más pequeños creados al ramificar más este subproblema y quizá después al ramificarlo más en “generaciones” subsecuentes). Lo que es más, a medida que se encuentren nuevas soluciones de apoyo con valores más grandes de Z^* será más fácil *sondear* de esta manera.

La tercera forma de sondeo es bastante directa. Si el método símplex encuentra que el relajamiento de PL de un subproblema *no tiene soluciones factibles*, entonces el subproblema en sí *no debe tener soluciones factibles*, de forma que puede eliminarse (sondearse).

En los tres casos, se lleva a cabo la búsqueda de una solución óptima mediante la investigación de sólo aquellos subproblemas que es posible que tengan una mejor solución factible que la de apoyo actual.

Resumen de las pruebas de sondeo. Un subproblema se *sondea* (se descarta para su consideración posterior) si

Prueba 1: Su cota $\leq Z^*$,

o

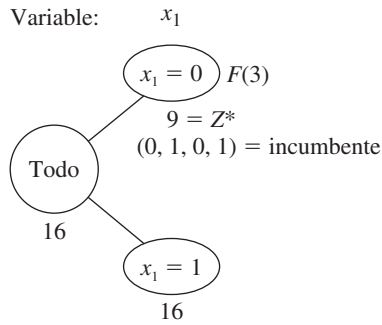
Prueba 2: Su relajamiento de PL no tiene soluciones factibles,

o

Prueba 3: La solución óptima para su relajamiento de PL es *entera*. (Si esta solución es mejor que la de apoyo, se convierte en la nueva solución de apoyo y se aplica de nuevo la prueba 1 a todos los subproblemas no sondeados, con la nueva mejor Z^* .)

En la figura 11.6 se resumen los resultados después de aplicar estas tres pruebas a los subproblemas 1 y 2; se muestra el *árbol de solución* actual. Sólo el subproblema 1 fue sondeado por la prueba 3, como lo indica $F(3)$ junto al nodo $x_1 = 0$. La solución de apoyo también se identifica debajo de este nodo.

■ **FIGURA 11.6**
Árbol de solución después
de la primera iteración del
algoritmo de ramificación
y acotamiento de PEB
para el ejemplo de la sec-
ción 11.1.



Las iteraciones siguientes ilustrarán la aplicación exitosa de las tres pruebas, pero antes de continuar con el ejemplo se resumirá el algoritmo que se aplica a este problema de PEB. (El algoritmo supone que la función objetivo se *maximizará*, que todos los coeficientes de la función objetivo son enteros y que, para efectos de simplicidad, el orden de las variables de la ramificación es $x_1, x_2, \dots, x_n$. Como se pudo observar anteriormente, gran parte de los algoritmos de ramificación y acotamiento utilizan complejos métodos para seleccionar las variables de ramificación.)

Resumen de la técnica de ramificación y acotamiento de PEB

Paso inicial: Se establece $Z^* = -\infty$. Se aplica al problema completo el paso de acotamiento, el de sondeo y la prueba de optimalidad que se describió. Si no se sondea, este problema se clasifica como el único “subproblema” restante para realizar la primera iteración completa.

Pasos de cada iteración:

1. *Ramificación:* Entre los subproblemas *restantes* (no sondeados) se elige el de creación *más reciente*. (Los empates se rompen con el que tenga la *cota más grande*.) Se ramifica este nodo de ese subproblema para crear dos nuevos subproblemas luego de establecer la siguiente variable (la variable de ramificación), ya sea en 0 o en 1.
2. *Acotamiento:* En cada nuevo subproblema, aplique el método *símplex* a su relajación LP para obtener una solución óptima que incluya el valor de Z para esta relajación LP. Si este valor de Z no es entero, redondéelo a un entero. (Si lo era, no es necesario ningún cambio.) Este valor entero de Z es el *acotado* para el subproblema.
3. *Sondeo:* A cada nuevo subproblema se le aplican las tres pruebas de sondeo que se resumieron antes y se descartan aquellos que son sondeados por cualquiera de las tres pruebas.

Prueba de optimalidad: El proceso termina cuando no existen subproblemas *restantes*; la solución de *apoyo* actual es óptima.⁴ De otra manera, se realiza otra iteración.

El paso de ramificación de este algoritmo merece un comentario respecto de la razón por la que se elige de esta manera el subproblema que se va a ramificar. Una opción que no se consideró sería elegir el subproblema restante con la *mejor cota*, puesto que este subproblema sería el más prometedor en cuanto a que puede contener la solución óptima del problema completo. La razón para usar la opción de seleccionar el subproblema de *creación más reciente* es que los *relajamientos* de PL se resuelven en el paso de acotamiento. En lugar de iniciar el método *símplex* cada vez desde el principio, cada relajamiento de PL en los problemas de gran escala se puede resolver casi siempre por *reoptimización*. Esta reoptimización implica revisar la tabla *símplex* final de relajamiento de PL anterior, según sea necesario, debido a las pocas diferencias en el modelo (igual que para el análisis de sensibilidad), y después aplicar unas cuantas iteraciones, quizá del método *símplex* dual. La reoptimización tiende a ser *mucho* más rápida que comenzar desde el principio, *siempre* que el

⁴ Si no existe una solución de apoyo o incumbente, la conclusión es que el problema no tiene soluciones factibles.

modelo anterior y el actual sean parecidos. Los modelos tenderán a parecerse si se usa la regla de ramificación descrita, pero *no* cuando van de un lado a otro en el árbol de solución, como cuando se elige el subproblema con la mejor cota.

Terminación del ejemplo

El patrón de las iteraciones que faltan será muy similar al de la primera iteración que se acaba de describir, excepto por la forma en que se realiza el sondeo. Por tanto, se resumirán los pasos de ramificación y acotamiento y se hará hincapié en el paso de sondeo.

Iteración 2. El único subproblema que queda corresponde al nodo $x_1 = 1$ en la figura 11.6, por lo que se ramificará este nodo para crear dos nuevos subproblemas.

Subproblema 3:

Se fija $x_1 = 1, x_2 = 0$ de manera que el subproblema que resulta es

$$\text{Maximizar} \quad Z = 9 + 6x_3 + 4x_4,$$

sujeta a

- (1) $5x_3 + 2x_4 \leq 4$
- (2) $x_3 + x_4 \leq 1$
- (3) $x_3 \leq 1$
- (4) $x_4 \leq 0$
- (5) x_j es binaria, para $j = 3, 4$.

Subproblema 4:

Se fija $x_1 = 1, x_2 = 1$ de manera que el subproblema que resulta es

$$\text{Maximizar} \quad Z = 14 + 6x_3 + 4x_4,$$

sujeta a

- (1) $5x_3 + 2x_4 \leq 1$
- (2) $x_3 + x_4 \leq 1$
- (3) $x_3 \leq 1$
- (4) $x_4 \leq 1$
- (5) x_j es binaria, para $j = 3, 4$.

Los relajamientos de PL de estos subproblemas se obtienen consumando la restricción adicional que se muestra a continuación de la versión relajada de la restricción (5). Sus soluciones óptimas se muestran también a continuación.

Relajamiento de PL del subproblema 3: (5) $x_1 \geq 1, x_2 \leq 0$ y $0 \leq x_j \leq 1$
para $j = 1, 2, 3, 4$.

Solución óptima: $(x_1, x_2, x_3, x_4) = \left(1, 0, \frac{4}{5}, 0\right)$ con $Z = 13\frac{4}{5}$,

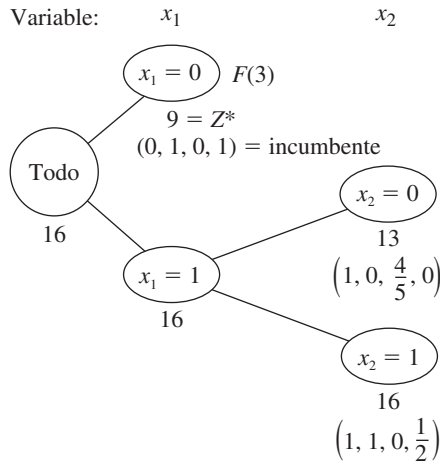
Relajamiento de PL del subproblema 4: (5) $x_1 \geq 1, x_2 \geq 1$ y $0 \leq x_j \leq 1$
para $j = 1, 2, 3, 4$.

Solución óptima: $(x_1, x_2, x_3, x_4) = \left(1, 1, 0, \frac{1}{2}\right)$ con $Z = 16$.

Las cotas que resultan para los subproblemas son

Cota del subproblema 3: $Z \leq 13$,

Cota del subproblema 4: $Z \leq 16$.



■ FIGURA 11.7

Árbol de solución después de la iteración 2 del algoritmo de ramificación y acotamiento de PEB del ejemplo de la sección 11.1.

Observe que estas cotas son más grandes que $Z^* = 9$, por lo que la prueba de sondeo 1 fracasa en ambos casos. La prueba 2 también fracasa, puesto que ambos relajamientos de PL tienen soluciones factibles (como lo indica la existencia de una solución óptima). De la misma manera, la prueba 3 fracasa porque ambas soluciones óptimas incluyen variables con valores no enteros.

En la figura 11.7 se muestra el árbol de solución resultante en este punto. La falta de una F a la derecha de cualquiera de los nuevos nodos indica que ambos quedan sin sondear.

Iteración 3. Hasta aquí, el algoritmo creó cuatro subproblemas. El subproblema 1 fue sondeado y el subproblema 2 se substituyó por (se dividió en) los subproblemas 3 y 4, pero estos dos quedan bajo consideración. Como fueron creados al mismo tiempo y el subproblema 4 ($x_1 = 1, x_2 = 1$) tiene una *cota* más grande ($16 > 13$), la siguiente ramificación se hace desde el nodo $(x_1, x_2) = (1, 1)$ en el árbol de solución, lo que crea los siguientes subproblemas nuevos (en donde la restricción 3 desaparece porque no contiene a x_4).

Subproblema 5:

Se fija $x_1 = 1, x_2 = 1, x_3 = 0$ y el subproblema que resulta es

$$\text{Maximizar} \quad Z = 14 + 4x_4,$$

sujeta a

- (1) $2x_4 \leq 1$
- (2), (4) $x_4 \leq 1$ (dos veces)
- (5) x_4 es binaria.

Subproblema 6:

Se fija $x_1 = 1, x_2 = 1, x_3 = 1$ de manera que el subproblema que resulta es

$$\text{Maximizar} \quad Z = 20 + 4x_4,$$

sujeta a

- (1) $2x_4 \leq -4$
- (2) $x_4 \leq 0$
- (4) $x_4 \leq 1$
- (5) x_4 es binaria.

Las relajaciones PL correspondientes tienen la versión relajada de la restricción (5), la solución óptima y la acotada (cuando existe) que se muestra en seguida.

Relajamiento de PL del subproblema 5:

$$(5) \quad x_1 \geq 1, \quad x_2 \geq 1, \quad x_3 \leq 0 \quad y \quad 0 \leq x_j \leq 1 \\ \text{para } j = 1, 2, 3, 4.$$

$$\text{Solución óptima: } (x_1, x_2, x_3, x_4) = \left(1, 1, 0, \frac{1}{2}\right), \quad \text{con } Z = 16.$$

$$\text{Cota: } Z \leq 16$$

Relajamiento de PL del subproblema 6:

$$(5) \quad x_1 \geq 1, \quad x_2 \geq 1, \quad x_3 \geq 1 \quad y \quad 0 \leq x_j \leq 1 \\ \text{para } j = 1, 2, 3, 4.$$

Solución óptima:

Ninguna, puesto que no hay soluciones factibles.

Cota: ninguna

En ambos subproblemas, la versión relajada de la restricción (5) tiene el efecto de fijar los valores de x_1 , x_2 y x_3 en los deseados y, por tanto, requieren que $0 \leq x_4 \leq 1$. Por ello, los relajamientos PL de estos subproblemas se reducen a las premisas de los subproblemas que se plantearon antes, excepto por el reemplazo de la restricción (5) por $0 \leq x_4 \leq 1$. Si se reducen estas relajaciones PL a problemas de una variable (más los valores fijos de x_1 , x_2 y x_3) se hace más sencillo ver que la solución óptima de la relajación PL del subproblema 5 es, en realidad, el que se proporcionó antes. De manera similar, observe que la combinación de la restricción 1 y $0 \leq x_4 \leq 1$ en el relajamiento de PL del subproblema 6 evita las soluciones factibles. Por tanto, este subproblema se sondea mediante la prueba 2. No obstante, el subproblema 5 no pasa esta prueba, ni la 1 ($16 > 9$), ni la prueba 3 ($x_4 = \frac{1}{2}$ no es un entero), así que permanece bajo consideración.

En la figura 11.8 se muestra el árbol de solución.

Iteración 4. Los subproblemas que corresponden a los nodos (1, 0) y (1, 1, 0) de la figura 11.8 permanecen bajo consideración, pero el último nodo es el de creación más reciente, por lo que se selecciona para ramificación. Como la variable de ramificación x_4 es la *última* variable, si se fija su valor en 0 o en 1, se crea *una solución* y no un subproblema que requiera más investigación. Las soluciones creadas son

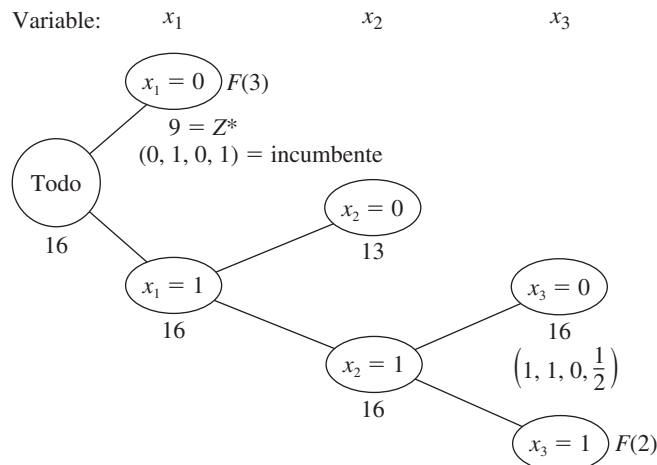
$$x_4 = 0: \quad (x_1, x_2, x_3, x_4) = (1, 1, 0, 0) \text{ es factible, con } Z = 14,$$

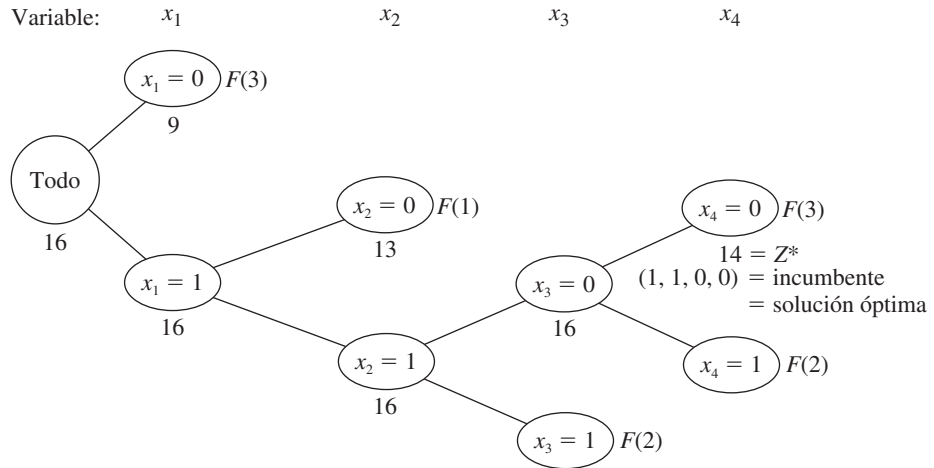
$$x_4 = 1: \quad (x_1, x_2, x_3, x_4) = (1, 1, 0, 1) \text{ es factible.}$$

Mediante la aplicación formal de las pruebas de sondeo, se comprueba que la primera solución pasa la prueba 3 y la segunda la prueba 2. Además, la primera solución factible es mejor que la de apoyo ($14 > 9$), por lo cual se convierte en la nueva solución de apoyo, con $Z^* = 14$.

■ FIGURA 11.8

Árbol de solución después de la iteración 3 del algoritmo de ramificación y acotamiento de PEB del ejemplo de la sección 11.1.





■ FIGURA 11.9

Árbol de solución después de la última (cuarta) iteración del algoritmo de ramificación y acotamiento de PEB del ejemplo de la sección 11.1.

Como se encontró una nueva solución de apoyo, se vuelve a aplicar la prueba 1 con el nuevo valor más grande de Z^* al único subproblema que queda, el del nodo (1, 0).

Subproblema 3:

$$\text{Cota} = 13 \leq Z^* = 14.$$

Entonces, este subproblema queda sondeado.

Se tiene ahora el árbol de solución de la figura 11.9. Observe que *no* hay subproblemas *restantes* (sin sondear). En consecuencia, la prueba de optimalidad indica que la solución de apoyo actual

$$(x_1, x_2, x_3, x_4) = (1, 1, 0, 0)$$

es óptima, y el problema termina aquí.

El OR Tutor incluye **otro ejemplo** de la aplicación de este algoritmo. También una rutina interactiva para ejecutar el algoritmo. Como es usual, los archivos de Excel, LINGO/LINDO y MPL/CPLEX para este capítulo muestran cómo se aplican las versiones para estudiantes de estos paquetes a los ejemplos del capítulo. Los algoritmos que se usan para solucionar los problemas de PEB son similares a los que se describen aquí.⁵

Otras opciones con la técnica de ramificación y acotamiento

En esta sección se ha ilustrado la técnica de ramificación y acotamiento con la descripción de un algoritmo básico para resolver problemas de programación entera binaria. Sin embargo, el marco general de trabajo de la técnica de ramificación y acotamiento proporciona una gran flexibilidad para diseñar un algoritmo específico para cualquier tipo de problema. Se dispone de muchas opciones y el diseño de un algoritmo eficiente requiere una adaptación para que se ajuste a la estructura particular del tipo de problema.

Cada algoritmo de ramificación y acotamiento contiene los mismos pasos básicos de *ramificación*, *acotamiento* y *sondeo*. La flexibilidad consiste en cómo se realizan estos pasos.

La *ramificación* siempre implica *seleccionar* un subproblema restante y *dividirlo* en subproblemas más pequeños. En este caso, la flexibilidad se encuentra en las reglas para seleccionar y dividir. El algoritmo seleccionado elige el subproblema de *creación más reciente*, porque este método es muy eficiente para *reoptimizar* cada relajamiento de PL a partir de la anterior. La otra regla popular es elegir el subproblema con la *mejor cota* puesto que tiende a llegar más rápido a mejores

⁵ En la versión profesional de LINGO, LINDO y CPLEX, el algoritmo de PEB utiliza también una variedad de técnicas elaboradas similares a las descritas en la sección 11.8.

soluciones de apoyo y más sondeos. También se puede usar una combinación de estas dos reglas. La *división*, por lo general (pero no siempre) se hace seleccionando una *variable de ramificación* a la cual se asignan ya sea valores individuales (como en el algoritmo de PEB), o intervalos de valores (como en el algoritmo de la siguiente sección). Algunos algoritmos más elaborados utilizan una regla para elegir de manera estratégica una variable de ramificación que conduzca a sondeos rápidos. Este enfoque es mucho más eficiente que la regla que utiliza nuestro algoritmo BP de seleccionar simplemente las variables de ramificación en su orden natural, $x_1, x_2, \dots, x_n$. Por ejemplo, una desventaja importante de esta simple regla para seleccionar la variable de ramificación es que si la variable tiene un valor entero en la solución óptima para la relajación PL del subproblema que se está ramificando, el subproblema siguiente que fija esta variable a este mismo valor entero también tendrá la misma solución óptima para su relajación PL, por lo que no se habrá avanzado nada hacia el sondeo. Por tanto, las opciones más estratégicas para seleccionar la variable de ramificación deben hacer algo parecido a seleccionar la variable cuyo valor en la solución óptima para la relajación PL del subproblema actual esté *más lejos* de ser un entero.

El *acotamiento* casi siempre se hace mediante la resolución de un *relajamiento*. Sin embargo, existen muchas formas de ellos. Por ejemplo, considere el **relajamiento de Lagrange**, en donde se *elimina* el conjunto completo de restricciones funcionales, en notación matricial $\mathbf{Ax} \leq \mathbf{b}$ (excepto quizá algunas restricciones “convenientes”), y después la función objetivo

$$\text{Maximizar} \quad Z = \mathbf{cx},$$

se reemplaza por

$$\text{Maximizar} \quad Z_R = \mathbf{cx} - \boldsymbol{\lambda}(\mathbf{Ax} - \mathbf{b}),$$

donde el vector fijo es $\boldsymbol{\lambda} \geq \mathbf{0}$. Si $\mathbf{x}^*$ es una solución óptima para el problema original, su $Z \leq Z_R$, por lo que cuando se obtiene un valor óptimo de Z_R con el relajamiento de Lagrange, se llega a una *cota* válida. Si $\boldsymbol{\lambda}$ se elige bien, esta cota tiende a ser razonablemente cerrada (al menos comparable con la cota del relajamiento de PL). Sin restricciones funcionales, este relajamiento se puede resolver muy rápido. Las desventajas son que las pruebas de sondeo 2 y 3 (revisadas) no son tan poderosas como las del relajamiento de PL.

En términos generales, las dos características que se buscan al elegir un relajamiento son que se pueda resolver relativamente rápido y que proporcione una cota bastante cerrada. Ninguna de las dos características por sí sola es adecuada. El relajamiento de PL se usa más porque proporciona un excelente trueque entre estos dos factores.

Una opción que se emplea en ciertas ocasiones es la solución rápida de un relajamiento y después, si no se logra el sondeo, se cierra el relajamiento de alguna forma para obtener una mejor cota.

El *sondeo* casi siempre se hace como se describió en el algoritmo de PEB. Los tres criterios para sondear se pueden establecer en términos más generales como sigue.

Resumen de criterios de sondeo. Un subproblema se *sondea* si el análisis de su *relajamiento* revela que

Criterio 1: Las soluciones factibles del problema cumplen con que $Z \leq Z^*$, o

Criterio 2: El subproblema no tiene soluciones factibles, o

Criterio 3: Se encontró una solución óptima del subproblema.

Igual que en el caso del algoritmo de PEB, los dos primeros criterios se aplican cuando se resuelve un relajamiento para obtener una cota sobre el subproblema y después verificar si esta cota es $\leq Z^*$ (prueba 1) o si el relajamiento no tiene soluciones factibles (prueba 2). Si el relajamiento difiere del subproblema *sólo* en que se eliminaron (o se relajaron) algunas restricciones, se aplica el tercer criterio para verificar si la solución óptima del relajamiento es *factible* para el subproblema, en cuyo caso debe ser *óptima* para él. En otros relajamientos (como el de Lagrange), se requiere un análisis adicional para determinar si la solución óptima para el relajamiento es óptima para el subproblema.

Si el problema original involucra *minimización* en lugar de maximización, se tienen dos opciones. Una es convertirlo a maximización en la forma usual (vea la sección 4.6). La otra es con-

vertir directamente el algoritmo de ramificación y acotamiento a la forma de minimización, lo que requiere cambiar la dirección de la desigualdad de la prueba de sondeo 1, de

¿La cota del subproblema es $\leq Z^*$?

a

¿La cota del subproblema es $\geq Z^*$?

Cuando se usa esta última desigualdad, si el valor de Z de la solución óptima para la relajación LP del subproblema no es entero, se redondea a un entero con el fin de obtener la cota del subproblema.

Hasta ahora se ha descrito cómo usar la técnica de ramificación y acotamiento para encontrar una solución óptima. Sin embargo, en el caso de empates a veces es deseable identificar *todas* estas soluciones óptimas para que la elección final entre ellas pueda hacerse sobre factores intangibles que no se incorporaron al modelo matemático. Para encontrar todas es necesario introducir unos cuantos cambios al procedimiento. Primero, se cambia la desigualdad débil en la prueba de sondeo 1 (¿La cota del subproblema es $\leq Z^*$?) por una desigualdad estricta (¿La cota del subproblema es $< Z^*$?) para que el sondeo no ocurra si el subproblema puede tener una solución factible *igual* a la de apoyo. Segundo, si la prueba de sondeo 3 pasa y la solución óptima para el subproblema tiene $Z = Z^*$, se almacena como *otra* solución de apoyo (empatada). Tercero, si la prueba de sondeo 3 proporciona una nueva solución de apoyo (empatada o no), se verifica si la solución óptima que se obtuvo para el *relajamiento* es *única*. Si no lo es, se identifican las otras soluciones óptimas del relajamiento y se verifica que sean óptimas también para el subproblema, en cuyo caso se convierten en soluciones de apoyo. Por último, cuando la *prueba de optimalidad* encuentra que no hay subconjuntos *restantes* (sin sondear), *todas* las soluciones de *apoyo* actuales serán soluciones *óptimas*.

Para terminar, debe hacerse notar que más que encontrar una solución óptima, la técnica de ramificación y acotamiento se puede usar también para encontrar una solución *cercana a la óptima*, en general, con mucho menos esfuerzo computacional. En algunas aplicaciones, una solución es “bastante buena” si su Z está lo “suficientemente cerca” del valor de Z de la solución óptima (llámese Z^{**}). *Suficientemente cerca* se puede definir como

$$Z^{**} - K \leq Z \quad \text{o como} \quad (1 - \alpha)Z^{**} \leq Z$$

para una constante (positiva) específica K o α . Por ejemplo, si se elige la segunda definición y $\alpha = 0.05$, se requiere que la solución se encuentre a 5% o menos de la óptima. En consecuencia, si se supiera que el valor de Z de la solución de apoyo actual (Z^*) satisface una de las dos desigualdades

$$Z^{**} - K \leq Z^* \quad \text{o} \quad (1 - \alpha)Z^{**} \leq Z^*$$

entonces el procedimiento podría terminar de inmediato con la elección de la solución de apoyo como la solución cercana a la óptima deseada. Aunque, de hecho, el procedimiento no identifica una solución óptima y la Z^{**} correspondiente, si esta solución (desconocida) es factible (y, por tanto, óptima) para el subproblema actual bajo investigación, entonces la prueba de sondeo 1 encuentra una cota superior tal que

$$Z^{**} \leq \text{cota}$$

de manera que

$$\text{Cota} - K \leq Z^* \quad \text{o} \quad (1 - \alpha)\text{cota} \leq Z^*$$

lo que implicaría que se satisface la desigualdad correspondiente en la afirmación anterior. Aun cuando esta solución no es factible para el subproblema actual, se obtiene una cota superior válida para el valor de Z de la solución óptima de ese subproblema. Así, es suficiente satisfacer cualquiera de las dos desigualdades para sondear este subproblema porque la solución de apoyo debe estar “bastante cerca” de la solución óptima del subproblema.

Por tanto, para encontrar una solución que esté bastante cerca de la óptima, sólo se necesita un cambio en el algoritmo de ramificación y acotamiento. Este cambio es sustituir la prueba de sondeo 1 en el subproblema

$$\text{Cota} \leq Z^*?$$

por

$$\text{Cota} - K \leq Z^*?$$

o bien

$$(1 - \alpha)(\text{cota}) \leq Z^*?$$

y realizar esta prueba *después* de la prueba 3 (para que una solución factible que se encuentra con $Z > Z^*$ se conserve como la nueva solución de apoyo). La razón para que esta prueba 1 más débil sea suficiente es que no importa qué tan cerca de la cota del subproblema se encuentre Z de la solución óptima (desconocida) del subproblema, la solución de apoyo todavía estará bastante cerca de esta solución (si se cumple la nueva desigualdad) como para que no sea necesario seguir analizando el subproblema. Cuando no hay subproblemas restantes, la solución de apoyo actual será la solución *cercana a la óptima* que se desea. Sin embargo, es mucho más sencillo sondear con esta nueva prueba (en cualquiera de sus formas), por lo que el algoritmo trabajará más rápido. Si se trata de problemas muy grandes, esta aceleración puede significar la diferencia entre terminar con una solución que sea cercana a la óptima y nunca terminar. En el caso de problemas muy grandes que surgen en la práctica, como el modelo brinda sólo una representación idealizada del problema real, es suficiente para todo propósito en la práctica encontrar una solución casi óptima para el modelo de esta forma. Por ende, en la práctica, se utiliza este atajo muy frecuentemente.

11.7 ALGORITMO DE RAMIFICACIÓN Y ACOTAMIENTO PARA PROGRAMACIÓN ENTERA MIXTA

A continuación se estudiará el problema general de programación entra mixta, PEM, donde *algunas* variables (por ejemplo, I de ellas) están restringidas a valores enteros (pero no necesariamente sólo 0 y 1), y el resto son variables continuas comunes. Por conveniencia en la notación, estas variables se ordenarán de manera que las primeras I de ellas sean variables con *restricción de enteros*. La forma general del problema que se va a estudiar es

$$\text{Maximizar} \quad Z = \sum_{j=1}^n c_j x_j,$$

sujeta a

$$\sum_{j=1}^n a_{ij} x_j \leq b_i, \quad \text{para } i = 1, 2, \dots, m,$$

y

$$\begin{aligned} x_j &\geq 0, & \text{para } j = 1, 2, \dots, n, \\ x_j &\text{ es entero,} & \text{para } j = 1, 2, \dots, I; I \leq n. \end{aligned}$$

(Cuando $I = n$, este problema se convierte en uno de PE pura.)

Se describirá un algoritmo básico de ramificación y acotamiento para resolver este problema que, un poco más elaborado, ha proporcionado el enfoque estándar a la PEM. La estructura de este algoritmo fue desarrollada por R. J. Dakin,⁶ con base en un algoritmo anterior que se debe a A. H. Land y A. G. Doig.⁷

La estructura de este algoritmo es bastante parecida a la del algoritmo de PEB que se presentó en la sección anterior. De nuevo la solución de un *relajamiento de PL* proporciona la base tanto para el paso de *acotamiento* como para el de *sondeo*. En realidad, sólo se necesitan cuatro cambios al algoritmo de PEB para manejar la generalización de variables enteras *binarias generales* y de PE *pura* a PE *mixta*.

⁶R. J. Dakin, "A Tree Search Algorithm for Mixed Integer Programming Problems", en *Computer Journal*, **8**(3): 250-255, 1965.

⁷A. H. Land y A. G. Doig, "An Automatic Method of Solving Discrete Programming Problems", en *Econometrica*, **28**: 497-520, 1960.

Un cambio involucra la elección de la *variable de ramificación*. Antes se elegía de manera automática la *siguiente* variable en el orden natural $(x_1, x_2, \dots, x_n)$. Ahora, las únicas variables que se toman en cuenta son las variables con *restricción de enteros* que tienen un valor *no entero* en la solución óptima del relajamiento de PL para el subproblema actual. La regla para elegir entre estas variables será seleccionar la *primera* en el orden natural. (Los paquetes de producción casi siempre utilizan una regla más complicada.)

El segundo cambio se refiere a los valores asignados a la variable de ramificación para crear los nuevos subproblemas. Antes, la variable *binaria* se fijaba en 0 y 1, respectivamente, para los dos nuevos subproblemas. Ahora, la variable *general* con restricción de entera puede tener un número grande de valores enteros posibles, y sería ineficiente crear y analizar *muchos* subproblemas mediante la determinación de cada valor entero de la variable. Entonces, se crean sólo *dos* nuevos subproblemas (como antes) al especificar *dos intervalos* de valores de la variable.

Para aclarar cómo se procede, sea x_j la variable de ramificación actual y x_j^* su valor (no entero) en la solución óptima del relajamiento de PL del subproblema actual. Se usan corchetes para denotar

$$[x_j^*] = \text{entero más grande} \leq x_j^*,$$

por lo cual, para los intervalos de valores de los dos nuevos subproblemas se tiene

$$x_j \leq [x_j^*] \quad \text{y} \quad x_j \geq [x_j^*] + 1,$$

respectivamente. Cada desigualdad se convierte en una *restricción adicional* del nuevo subproblema. Por ejemplo, si $x_j^* = 3\frac{1}{2}$, entonces

$$x_j \leq 3 \quad \text{y} \quad x_j \geq 4$$

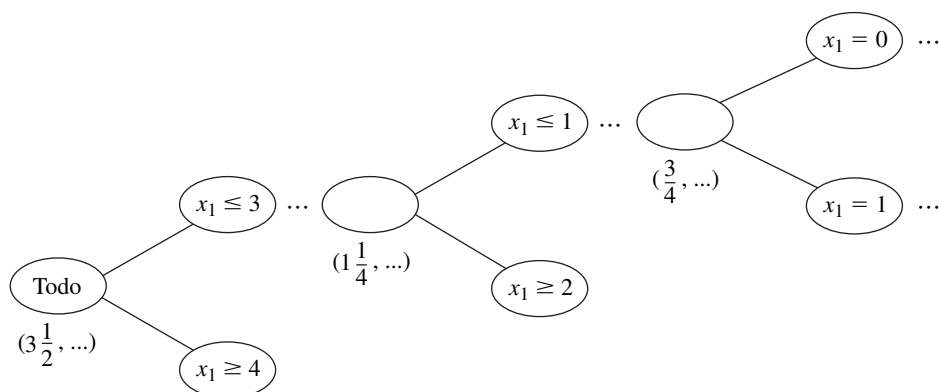
son las restricciones adicionales respectivas de los dos nuevos subproblemas.

Cuando se combinan los dos cambios al algoritmo de PEB que se acaba de describir, puede ocurrir un interesante fenómeno de una *variable de ramificación recurrente*. Para ilustrarla, como se muestra en la figura 11.10, sea $j = 1$ en el ejemplo anterior donde $x_1^* = 3\frac{1}{2}$ y considere el nuevo subproblema con $x_1 \leq 3$. Cuando se resuelve el relajamiento de PL para un descendiente de este subproblema se obtiene $x_1^* = 1\frac{1}{4}$. Entonces, x_1 *recurre* como variable de ramificación y los dos nuevos subproblemas creados tienen la restricción adicional $x_1 \leq 1$ y $x_1 \geq 2$, respectivamente (así como la restricción adicional anterior, $x_1 \leq 3$). Más adelante, cuando se resuelva el relajamiento de PL de un descendiente de, por ejemplo, $x_1 \leq 1$, se supondrá que $x_1^* = \frac{3}{4}$. Entonces x_1 *recurre* de nuevo como variable de ramificación, y los dos nuevos subproblemas creados tienen $x_1 = 0$ (debido a la nueva restricción $x_1 \leq 0$ y la restricción de no negatividad sobre x_1) y $x_1 = 1$ (debido a la nueva restricción $x_1 \geq 1$ y la anterior $x_1 \leq 1$).

El tercer cambio se hace en el *paso de acotamiento*. Antes, con un problema de PE *pura* y coeficientes enteros en la función objetivo, el valor de Z de la solución óptima del relajamiento de PL del subproblema se *redondeaba hacia abajo* para obtener la cota, puesto que cualquier solución factible del problema debía tener una Z *entera*. Ahora, con algunas variables *sin* la restricción de integridad, la cota es el valor de Z *sin* redondear.

■ FIGURA 11.10

Ilustración del fenómeno de una *variable de ramificación recurrente*, donde x_1 se convierte en una variable de ramificación tres veces por tener un valor no entero en la solución óptima del relajamiento de PL en tres nodos.



Recuadro de aplicación

Waste Management, Inc. (una de las 100 empresas de Fortune), con casa matriz en Houston, Texas, es un proveedor líder de servicios de manejo de basura en Norteamérica. Su red de operaciones está formada por 293 sitios para depósito de basura, 16 plantas de conversión de basura en energía, 72 instalaciones para la conversión de gas en energía, 146 plantas de reciclaje, 346 estaciones de transferencia y 435 plantas de recolección (depósitos) para brindar servicios a casi 200 millones de clientes residenciales y 2 millones de clientes comerciales en Estados Unidos y Canadá.

Es necesario que los vehículos de recolección y transferencia de la compañía recorran casi 20 000 rutas todos los días. Con un costo de operación anual de casi 120 000 dólares por vehículo, la gerencia deseaba contar con un sistema de administración de rutas que fuera capaz de maximizar la eficiencia de cada una de las rutas. Por tanto, se formó un grupo de IO con el fin de atacar este problema.

La parte medular del sistema de administración de rutas que desarrolló dicho grupo es un enorme *modelo BIP mezclado* que optimiza las rutas asignadas a los respectivos vehículos de recolección y transferencia. Como la función objetivo toma en cuenta varios factores, el objetivo principal consiste

en minimizar el tiempo total de viaje. Las variables de decisión principales son variables binarias iguales a 1 si la ruta asignada a un vehículo particular incluye una posible variación e iguales a 0 de otra forma. Un sistema de información geográfica (GIS) proporciona información acerca de la distancia y el tiempo que se requieren para recorrer cualquier par de puntos. Todo esto se encuentra incluido en la aplicación Java basada en la Web que está integrada en los demás sistemas de la compañía.

Se estima que la implementación reciente de este sistema de administración de rutas *incrementará el flujo de efectivo de la compañía en 648 millones de dólares* en un periodo de 5 años, en gran medida debido a los ahorros del orden de **498 millones de dólares** en gastos de operación en el mismo periodo. Asimismo, brinda un mejor servicio a los clientes.

Fuente: S. Sahoo, S. Kim, B. I. Kim, B. Krass y A. Popov, Jr.: "Routing Optimization for Waste Management", en *Interfaces*, 35(1): 24-36, enero-febrero de 2005. (En el sitio en internet de este libro —www.mhhe.com/hillier— se puede encontrar una liga hacia este artículo.)

El cuarto (y último) cambio al algoritmo de PEB para obtener el algoritmo de PEM se refiere a la prueba de sondeo 3. Antes, con un problema de PE *pura*, la prueba consistía en que la solución al relajamiento de PL del subproblema fuera *entera*, puesto que ello aseguraba una solución factible y, por tanto, óptima del subproblema. Ahora, con un problema de PE *mixta*, la prueba requiere que sólo las variables *restringidas a enteras* tengan valores *enteros* en la solución óptima del relajamiento de PL del subproblema, puesto que esto es suficiente para asegurar que la solución es factible y, por tanto, óptima para el subproblema.

La incorporación de estos cuatro cambios al resumen que se presentó en la sección anterior sobre el algoritmo de programación entera binaria conduce al siguiente resumen del nuevo algoritmo de programación entera mixta. (Como antes, este resumen supone que se va a maximizar la función objetivo, pero el único cambio necesario para la minimización consiste en cambiar la dirección de la desigualdad de la prueba de sondeo 1.)

Resumen del algoritmo de ramificación y acotamiento de PEM

Paso inicial: Se establece $Z^* = -\infty$. Se aplica el paso de acotamiento, el paso de sondeo y la prueba de optimalidad que se describe después del problema completo. Si no queda sondeado, se clasifica este problema como el único subproblema restante para realizar la primera iteración completa.

Pasos de cada iteración:

1. **Ramificación:** Entre los subproblemas *restantes* (no sondeados), se selecciona el de *creación más reciente*. (Los empates se rompen con la *cota más grande*.) Entre las variables *restringidas a enteros*, que tienen valores *no enteros* en la solución óptima del relajamiento de PL del subproblema, se elige la *primera* en el orden natural como la *variable de ramificación*. Si x_j es esta variable y x_j^* su valor en esta solución, se debe ramificar desde el nodo del subproblema para crear dos nuevos subproblemas luego de agregar las restricciones respectivas $x_j \leq [x_j^*]$ y $x_j \geq [x_j^*] + 1$.
2. **Acotamiento:** Se obtiene la cota de cada subproblema si se aplica el método *símplex* (o el método *símplex dual* si se reoptimiza) al relajamiento de PL y se utiliza el valor de Z para la solución óptima resultante.

3. *Sondeo*: Se aplican las pruebas de sondeo que se presentan a continuación a cada nuevo subproblema y se descartan aquellos que quedan sondeados por cualquiera de las pruebas.

Prueba 1: Su cota $\leq Z^*$, donde Z^* es el valor de Z en la solución de apoyo actual.

Prueba 2: Su relajamiento de PL no tiene soluciones factibles.

Prueba 3: La solución óptima para su relajamiento de PL tiene valores *enteros* en todas sus variables *restringidas a enteros*. (Si esta solución es mejor que la de apoyo, se convierte en la nueva solución de apoyo y se vuelve a aplicar la prueba 1 con la nueva Z^* a todos los subproblemas no sondeados.)

Prueba de optimalidad: El proceso se detiene cuando *no* hay subproblemas restantes; la solución de apoyo actual es óptima.⁸ De otra manera, se realiza otra iteración.

Ejemplo de PEM. Ahora se ilustrará este algoritmo mediante su aplicación al siguiente problema de PEM:

$$\text{Maximizar} \quad Z = 4x_1 - 2x_2 + 7x_3 - x_4,$$

sujeta a

$$x_1 + 5x_3 \leq 10$$

$$x_1 + x_2 - x_3 \leq 1$$

$$6x_1 - 5x_2 \leq 0$$

$$-x_1 + 2x_3 - 2x_4 \leq 3$$

y

$$x_j \geq 0, \quad \text{para } j = 1, 2, 3, 4$$

$$x_j \text{ es entero,} \quad \text{para } j = 1, 2, 3.$$

Observe que el número de variables restringidas a enteros es $I = 3$, de manera que x_4 es la única variable continua.

Paso inicial. Después de establecer $Z^* = -\infty$, se forma el relajamiento de PL de este problema mediante la *eliminación* del conjunto de restricciones si x_j es entero para $j = 1, 2, 3$. Si se aplica el método simplex a este relajamiento de PL, la solución óptima es

$$\text{Relajamiento de PL de todo el problema: } (x_1, x_2, x_3, x_4) = \left(\frac{5}{4}, \frac{3}{2}, \frac{7}{4}, 0\right), \quad \text{con } Z = 14\frac{1}{4}.$$

Como tiene soluciones *factibles* y esta solución óptima tiene valores *no enteros* para sus variables restringidas a enteros, se sondea todo el problema y el algoritmo continúa abajo con la primera iteración completa.

Iteración 1. En esta solución óptima del relajamiento de PL la *primera* variable restringida a enteros que tiene un valor no entero es $x_1 = \frac{5}{4}$, con lo cual ésta se convierte en la variable de ramificación. Cuando se ramifica desde el nodo de *todo* el problema (soluciones factibles de *todo*) con esta variable se crean los siguientes dos subproblemas:

Subproblema 1:

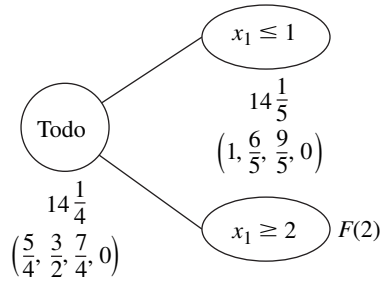
Problema original más la restricción adicional

$$x_1 \leq 1.$$

⁸ Si no hay solución de apoyo (incumbente), la conclusión es que el problema no tiene soluciones factibles.

■ FIGURA 11.11

Árbol de solución después de la primera iteración del algoritmo de ramificación y acotamiento de PEM del ejemplo.



Subproblema 2:

Problema original más la restricción adicional

$$x_1 \geq 2.$$

De nuevo se elimina el conjunto de restricciones a valores enteros y se resuelven los relajamientos de PL de estos dos subproblemas; los resultados son:

Subproblema 1:

Solución óptima para la relajación PL: $(x_1, x_2, x_3, x_4) = \left(1, \frac{6}{5}, \frac{9}{5}, 0\right)$, con $Z = 14\frac{1}{5}$.

Cota: $Z \leq 14\frac{1}{5}.$

Subproblema 2:

Relajación PL: Soluciones no factibles.

Este resultado del subproblema 2 significa que queda sondeado por la prueba 2. Igual que en el caso del problema completo, el subproblema 1 no pasa las pruebas de sondeo.

En la figura 11.11 se resumen todos estos resultados en el árbol de solución.

Iteración 2. Con sólo un subproblema restante que corresponde al nodo $x_1 \leq 1$ en la figura 11.11, la siguiente ramificación se hace desde ahí. Cuando se examina la solución óptima del relajamiento de PL que se presenta en seguida, se observa que este nodo revela que la *variable de ramificación* es x_2 , puesto que $x_2 = \frac{6}{5}$ es la primera variable restringida a enteros que no tiene un valor entero. Al agregar una de las restricciones, $x_2 \leq 1$ o $x_2 \geq 2$, se crean los dos nuevos subproblemas que siguen.

Subproblema 3:

Problema original más las restricciones adicionales

$$x_1 \leq 1, \quad x_2 \leq 1.$$

Subproblema 4:

Problema original más las restricciones adicionales

$$x_1 \leq 1, \quad x_2 \geq 2.$$

Al resolver sus relajamientos de PL se obtienen los siguientes resultados.

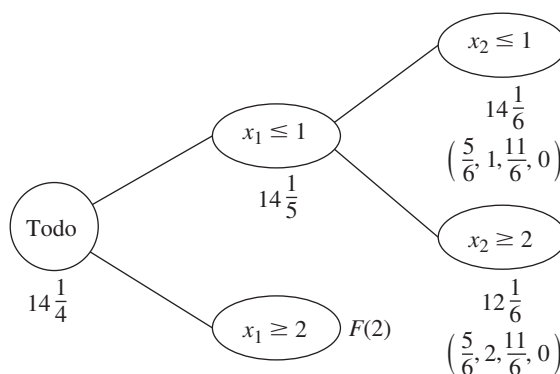
Subproblema 3:

Solución óptima de la relajación PL: $(x_1, x_2, x_3, x_4) = \left(\frac{5}{6}, 1, \frac{11}{6}, 0\right)$, con $Z = 14\frac{1}{6}$.

Cota: $Z \leq 14\frac{1}{6}.$

■ FIGURA 11.12

Árbol de solución después de la segunda iteración del algoritmo de ramificación y acotamiento de PEM del ejemplo.



Subproblema 4:

Solución óptima de la relajación LP: $(x_1, x_2, x_3, x_4) = \left(\frac{5}{6}, 2, \frac{11}{6}, 0\right)$, con $Z = 12\frac{1}{6}$.

Cota: $Z \leq 12\frac{1}{6}$.

Como ambas soluciones existen (soluciones factibles) y tienen valores no enteros de variables restringidas a enteros, ninguno de los subproblemas se sondea. (La prueba 1 no es operativa, puesto que todavía $Z^* = -\infty$, hasta que se encuentre la primera solución de apoyo.)

Hasta este punto, el árbol de solución se presenta en la figura 11.12.

Iteración 3. De los dos subproblemas restantes (3 y 4) que se crearon al mismo tiempo, se selecciona el que tiene la cota más grande (subproblema 3, con $14\frac{1}{6} > 12\frac{1}{6}$) para la siguiente ramificación. Como $x_1 = \frac{5}{6}$ tiene un valor no entero en la solución óptima del relajamiento de PL de su subproblema, x_1 se convierte en la variable de ramificación. (Observe que x_1 es ahora una variable de ramificación *recurrente*, pues también se seleccionó en la iteración 1.) Esto conduce a los siguientes subproblemas.

Subproblema 5:

Problema original más las restricciones adicionales

$$\begin{aligned} x_1 &\leq 1 \\ x_2 &\leq 1 \\ x_1 &\leq 0 \quad (\text{entonces } x_1 = 0). \end{aligned}$$

Subproblema 6:

Problema original más las restricciones adicionales

$$\begin{aligned} x_1 &\leq 1 \\ x_2 &\leq 1 \\ x_1 &\geq 1 \quad (\text{entonces } x_1 = 1). \end{aligned}$$

Los resultados que se obtienen cuando se resuelven los relajamientos de PL son los siguientes.

Subproblema 5:

Solución óptima de la relajación LP: $(x_1, x_2, x_3, x_4) = \left(0, 0, 2, \frac{1}{2}\right)$, con $Z = 13\frac{1}{2}$.

Cota: $Z \leq 13\frac{1}{2}$.

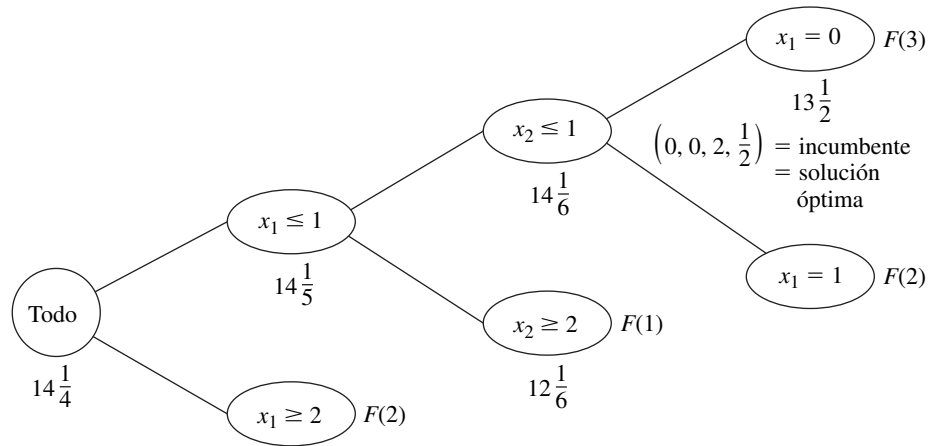
Subproblema 6:

Relajamiento LP

No tiene soluciones factibles.

■ FIGURA 11.13

Árbol de solución después de la última (tercera) iteración del algoritmo de ramificación y acotamiento de PEM del ejemplo.



El subproblema 6 se sondea de inmediato con la prueba 2. Sin embargo, observe que el subproblema 5 también se puede sondear. La prueba 3 pasa porque la solución óptima de este relajamiento de PL tiene valores enteros ($x_1 = 0$, $x_2 = 0$, $x_3 = 2$) para las tres variables restringidas a enteros. (No importa que $x_4 = \frac{1}{2}$, puesto que x_4 no está restringida a enteros.) Esta solución *factible* para el problema original se convierte en la primera solución de apoyo (incumbente):

$$\text{Incumbente} = \left(0, 0, 2, \frac{1}{2}\right) \quad \text{con } Z^* = 13\frac{1}{2}.$$

Con esta Z^* se vuelve a realizar la prueba de sondeo 1 al otro subproblema (subproblema 4) y pasa la prueba, puesto que su cota $12\frac{1}{6} \leq Z^*$.

Esta iteración tuvo éxito en el sondeo de las tres maneras posibles. Lo que es más, ya no hay subproblemas restantes; por tanto, la solución de apoyo actual es óptima.

$$\text{Solución óptima} = \left(0, 0, 2, \frac{1}{2}\right), \quad \text{con } Z = 13\frac{1}{2}.$$

Estos resultados se resumen en el árbol de solución de la figura 11.13.

En el OR Tutor se presenta **otro ejemplo** de aplicación del algoritmo de PEM. Además, en la sección Worked Examples en el sitio en internet de este libro se presenta un **ejemplo pequeño** (sólo dos variables, pero restringidas a enteros) que incluye presentaciones gráficas. En el IOR Tutorial se incluye también una rutina interactiva para ejecutar este algoritmo.

11.8 ENFOQUE DE RAMIFICACIÓN Y CORTE PARA RESOLVER PROBLEMAS DE PEB

La programación entera ha sido un área emocionante de IO durante los últimos años debido a los notables adelantos que se han logrado en su metodología de solución.

Antecedentes

Para dar una mejor perspectiva sobre este progreso, considere los antecedentes históricos. A finales de la década de 1960 y principios de la de 1970 se presentó un gran cambio con el desarrollo y perfeccionamiento del enfoque de ramificación y acotamiento. Después, todo siguió igual; se podían resolver en forma muy eficiente problemas relativamente pequeños (muy por debajo de las 100 variables) pero aún un pequeño aumento del tamaño del problema podía causar una explosión en el tiempo de cálculos más allá de los límites factibles. Se hacían pocos progresos para vencer